

队伍编号	MC25001429
题号	(C)

基于 SVM-NLMS 自适应滤波算法的音频高质量压缩与去噪研究

摘要

随着信息技术的快速发展，人们越来越关注音频文件的传输速率，而音频文件压缩体积虽然带来了文件大小下降，也导致了音质受损。因此如何取得音频文件压缩与音质保留之间的平衡，是一个值得研究和探索的问题。本文围绕音频文件高质量读写与去噪优化问题，综合运用熵权法、灰色关联法、SVM 支持向量机等模型对于音频压缩进行参数优化，并从众多音频去噪方法中选取归一最小均方算法（NLMS，一种自适应滤波算法）针对携带不同噪声类型的音频进行噪声识别与去噪，并验证其有效性。

对于问题 1，通过构建综合评估分数 AIS，量化音频格式在存储效率与音质保真度间的平衡，构建了包含文件大小；音质损失（信噪比 SNR）；编解码复杂度（CPU 占用率）；适用场景打分的四维评价模型。通过在不同的音频文件内容下分别赋权：语音场景下侧重压缩率（权重 0.35）和编解码速度（权重 0.25），音乐场景下侧重信噪比（权重 0.35）和音质适配性（权重 0.25）。选择 5 种常见音频格式进行分类并计算，发现 WAV 格式在音质优先场景下综合得分最高为 0.885，MP3 在语音传输场景中表现最优为 0.745，为格式选择提供了量化依据。

针对问题 2，基于附件 1 数据，建立采样率、比特率、文件大小的三维“性价比投影空间”，通过欧氏距离度量参数组合的优劣进而排序。语音场景中，AAC 格式（44100Hz, 96kbps）以文件大小 33.2KB、SNR 25.6dB 成为最优解；音乐场景中，MP3 格式（44100Hz, 64kbps）则因其均衡音质与压缩比 SNR 22.1dB，大小 78.6KB 位列第一。结合熵权法-灰色关联度分析指标的重要性与关联度，语音场景中文件大小与比特率关联度超 0.93，音乐场景中采样率对音质影响权重 0.32，为参数优化提供敏感因子参考。

承接上一问题，问题 3 设计了“频谱特征提取-SVM 分类-参数匹配”的自适应流程。对于文件的内容分类，首先提取其频谱图象并通过图像判断其内容；接着采用 SVM 二分类法，通过短时傅里叶变换（STFT）提取梅尔频率倒谱系数（MFCC）；梅尔频谱（Mel）；过零率（ZCR）特征，利用 RBF 作为核函数，实现语音/音乐二分类，测试集准确率达 96.15%。文件压缩要对参数进行选择，针对语音样本建议采用 AAC 格式（44100Hz, 96kbps）压缩后文件大小缩减 95%，而 AIS 评分上升 13%；音乐样本采用 MP3 格式（44100Hz, 128kbps）压缩后大小缩减 89%，AIS 评分上升 0.1，验证方案有效性。

围绕音频去噪，问题 4 中通过时频分析识别附件 2 中音频的噪声类型（part1.wav 含带状噪声与背景噪声，part2.wav 含较多突发噪声与白噪声），采用归一化最小均方（NLMS）算法实现动态降噪。针对不同噪声调整滤波器阶数（L）与步长（ μ ）：背景噪声场景取 $L = 256$ 、 $\mu = 0.1$ ，降噪后 SNR 从 12.3dB 提升至 25.6dB；突发噪声场景取 $L = 512$ 、 $\mu = 0.2$ ，SNR 从 8.7dB 提升至 22.1dB。结果表明，NLMS 算法对平稳噪声抑制效果显著，具有一定的去噪效果。

关键词：音频压缩评价；熵权法-灰色关联度；NLMS 自适应滤波；SVM；音频信噪比；多噪声去噪

目录

1 问题背景与重述	1
1.1 问题背景	1
1.2 问题重述	1
1.3 任务内容	2
2 模型说明	3
3 变量说明	3
4 问题解决	4
4.1 问题 1	4
4.1.1 评估指标建立	4
4.1.2 目标函数与权重设定	5
4.1.3 评价体系应用	6
4.2 问题 2	8
4.2.1 最优“性价比”组合的寻找与排序	8
4.2.2 指标关联度	10
4.2.3 音频最佳参数推荐	12
4.3 问题 3	13
4.3.1 识别音频内容	13
4.3.2 SVM 二分类法	17
4.3.3 最佳压缩参数设置	18
4.4 问题 4	20
4.4.1 识别不同噪音	20
4.4.2 自适应滤波算法 NLMS	20
5 总结与分析	24
5.1 模型优点与特色	24
5.2 可扩展之处	24
参考文献	25
附件 1 代码	26
附录 2 音频排序完整版	41

1 问题背景与重述

1.1 问题背景

随着数字媒体技术的快速发展，人们对于音视频娱乐的需求越来越高^[1]，具有良好传导性的音频传播技术越发重要。随着众多音频平台如网易云、QQ 音乐、喜马拉雅以及视频播放平台如 Bilibili 等平台的兴起，越来越多的研究趋向于针对音频的质量进行提升展开。如何保障音频传播的无损、高质量，不断地改进音频采样率等信息来提升音频的质量，满足听众的听觉享受。



图 1 主流音乐 APP 软件

然而，随着移动端设备数量的逐步上涨^[2]，手机、平板的出现将摄影、娱乐、游戏乃至音乐播放等功能融入到一个设备中，这不仅对于移动端设备的设计生产工艺与科技提出较高要求，同时还要各个音乐平台、视频平台对于其平台上的音频资源进行压缩。

对音频、视频进行压缩，主要有以下原因：（1）信息快速传播（网络带宽限制）；（2）存储空间限制；（3）传输过程中经历错误数据恢复；（4）达到兼容性和标准化；（5）数据的实时性要求；（6）数据加密与安全保护。然而对音频文件的压缩，也会导致音频出现音质下降的情况。目前主流的压缩方式主要分为有损压缩与无损压缩，这些压缩方式都是为了给音频文件“瘦身”，使其占据更小的存储空间。根据压缩音频的算法不同、适用的场景不同，应当充分考虑如何权衡音频文件体积与音频音质之间的关系，成为了一大研究重点。

1.2 问题重述

针对任务要求中的 4 个问题，我们将问题进行整理，并对此进行解读。

对于**问题 1**，需要综合以下四个因素：文件大小、音质损失、编解码复杂度以及适用场景。选择相应的可测、可量化数据来评估不同音频文件压缩方法的相关优劣，并于原始音频文件（未压缩版本）进行比较，进而得出求出一个关于音频文件所占空间、音质、压缩复杂度与适用性的目标函数，对其进行优化。

对于**问题 2**，将围绕问题中的目标函数，通过调整目标函数的相应参数，使得评估模型在音乐和语音不同内容的情形下，设置不同的参数，并按照现实情况，以模型不同参数的要求来选择不同的音频压缩方案，对于音频文件进行压缩。针对音乐和语音音频，需要选择不同的压缩方案，每个压缩方案也对应了一套参数。

针对**问题 3**，题目给出了一个原始语音音频与原始音乐音频，要求采取不同的编码方式。首先要判断音频内容是属于人声（语音）或是音乐，并选择在问题 2 中设置的参数，对于音频文件进行压缩，使得其改进后的文件所对应的目标函数结果相较于压缩前更优，要选取合适压缩参数。目标是在保证音质基本不受损失的情况下，对于音频进行压缩使其体积下降。

最后，围绕**问题 4**，需要建立数学模型，根据音频图谱识别出文件中的嘈杂声、噪声等等干扰声。结合前有研究，提出一种去噪音策略。针对不同的噪音类型采取不同的处理方法。结合可视化频谱图像与信噪比结果，讨论该套去噪音方案的可行性。

1.3 任务内容

(1) 建立优化模型：设立一个用于评估不同音频压缩方案的模型，其为一个优化函数，设置一系列权重来模拟不同情形下对于音频压缩的侧重点（例如语音音频侧重于压缩比，而音乐音频侧重于保障音质）。

(2) 设立性价比指标：涉及采样率、比特深度、压缩算法的影响因素，设立一个度量压缩性价比的指标。根据计算性价比，对于附件 1 文件夹中的音频进行评估和排序。

(3) 自适应编码方案：对于原始音频文件，需要对其进行压缩。根据压缩音频的内容不同，首先要对其进行判断，选择不同的参数代入优化模型，因而选择不同的压缩方法对其进行压缩，并求出其优化后的目标函数值，与压缩前结果比较。

(4) 降噪处理：选择一种自适应滤波器，调整其中的参数应对带有不同噪声类型的音频频谱。通过研究和识别噪声特征，对不同种类的噪声分类并针对性地设置参数，进而得到降噪后的音频。与去噪前的音频进行信噪比对比，验证滤波器的准确性。

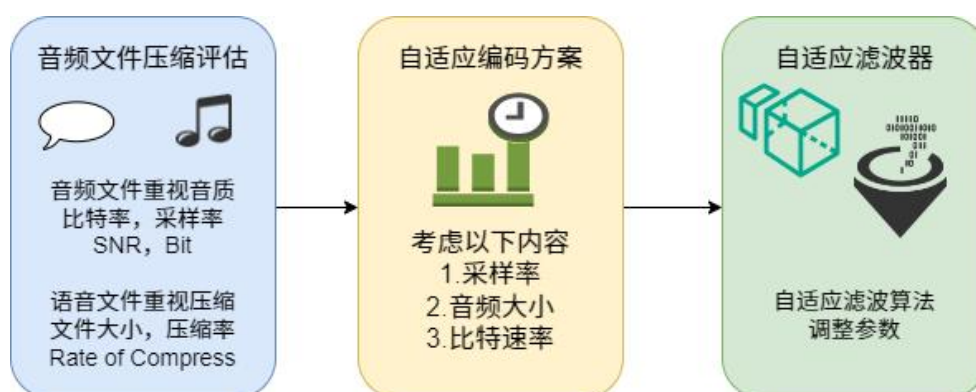


图 2 任务流程图

2 模型说明

为便于设计算法和计算，同时保证文章中模型的可行性与参考性，本文开展的工作以及得出的成果将基于以下说明。

- 假设说明 1：测试文件的编码解码复杂度需要考虑编码解码的时间，而这个时间过于短，而计算机性能与测试平台不同也会导致时间不同。本文中的编解码复杂度由 CPU 峰值占用率得到，文件编解码工作统一于一台电脑端设备。文中直接给出数据归一化后的结果。
- 假设说明 2：本文中的编解码以及相关代码涉及的需要对音频进行处理的工作均基于 ffmpeg (<https://ffmpeg.org/>) 对音频流媒体文件进行处理。

3 变量说明

表 1 变量说明表

变量	说明
$R_S(R_{normal})$	文件压缩比（归一化后的压缩比）
SNR	信噪比
MSE	均方误差
K	解码编码 CPU 占用率
α	情景系数（考虑语音或音乐音频设置不同）
S	特定场景适用性打分
X_{normal}	表示归一化后的数值， X 为指标
w_i	第 i 个指标的权重
W_c	在情况 c 下由 w_i 构成的权重系数组合
AIS	某一个 W_c 设定下的音频指标分数
(F, BR, SR)	文件大小，比特速率，采样率构成的三维点

4 问题解决

4.1 问题 1

对于问题 1，首先要建立一个关于音频的综合评价模型。本文中拟设计一个目标函数 Z ，并在模型中涉及四个方面：**文件大小**，**音质损失**，**编解码复杂度**以及**适用场景**。对于每一个评价指标，本文将选择相应的评估数据对其进行衡量，采用加权求和的方式得到目标 Z ，并**使之最大化**，求出 Z_{max} 。

4.1.1 评估指标建立

具体指标如下：

(1) 文件大小：反映音频文件压缩后的体积。在这里选择压缩率 R_S 作为文件压缩的效用衡量指标。其具体计算步骤如下：

$$R_S = \frac{S_{compress}}{S_{origin}}$$

在上述式子中， S_{origin} 、 $S_{compress}$ 分别表示压缩前、后的音频文件体积。通过比值可以计算得到压缩率，其反应了文件压缩的程度。

(2) 音质损失：音质损失代表了音频经过压缩处理后的音质保留情况，以信噪比/峰值信噪比来度量，其反映了声波信号的保留程度，或者声波信号在杂音信号与声波信号中的占比。目前中众多研究围绕各种场合中的相关噪音^[3-4]。本文中采用信噪比数据来进行衡量该指标，具体计算公式如下：

$$SNR = 10 \log_{10} \left(\frac{\sum_n x_{origin}^2(n)}{\sum_n (x_{origin}(n) - x_{compress}(n))^2} \right)$$

其中， $x_{origin}(n)$ 为原始音频信号， $x_{compress}(n)$ 为压缩后的信号。使用 librosa 库读取波形数据，在保证信号长度一致的情况下，计算其均方误差 MSE ：

$$MSE = \frac{1}{N} \sum_{n=0}^{N-1} (x_{origin}(n) - x_{compress}(n))^2$$

(3) 编解码复杂度：为了得出较为可靠的数据，由于编解码的时间过快，本文采用 CPU 峰值占用率作为编解码复杂度的度量指标。计算方式如下：

$$K = K_{code} + K_{decode}$$

其中， K_{code} 为编码 CPU 峰值占用率， K_{decode} 为解码 CPU 峰值占用率。该数据从统一设备编解码过程中得到，保证数据的参考意义。该项数据越低，表明编解码的时间、空间复杂度越低。

(4) 适用场景：不同的音频文件格式，有的是未压缩文件，有的是压缩文件。主要考虑到压缩带来的体积下降和音质受损，我们选择“音质优先”与“传输有限”两个场景，

每种音频文件所对应的分数 S 受到两个场景适用性的评分影响，在不同的场景下，侧重于不同的方面。分数 S 的计算方式如下：

$$S = \alpha S_1 + (1 - \alpha) S_2$$

上式中， α 代表情景系数，对于语音类音频（传输有限）与音乐类音频（音质有优先）分别选取不同的值，而 S_1, S_2 的取值需要依照压缩文件的算法、具体音频文件的使用情况以及文件格式是否具有较高的兼容性具体讨论。

4.1.2 目标函数与权重设定

上文中，对评估指标以及其四个方面的描述已然详尽，其具体计算方式也已给出。接下来的部分，将围绕目标函数的设立与参数设定方面进行论述。设立一系列权重系数来表示指标的重要性。

（1）指标归一化

对于上述得到的结果，需要对数据进行归一化处理。由于数据的单位和数量级不同，要消除其带来的影响，使得计算结果更为准确。按照指标分类成收益型与成本型指标，在本部分开头设定求目标函数极大化，因而需要将成本型指标转化为收益型。具体计算如下：

- 收益型指标： SNR, S （越大越好）

$$X_{normal} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- 成本型指标： K （越小越好）

$$X_{normal} = 1 - \frac{X - X_{min}}{X_{max} - X_{min}} = \frac{X_{max} - X}{X_{max} - X_{min}}$$

- 其他处理方式： R_S （取倒数后作为收益型指标）

$$R = \frac{1}{R_S}$$

（2）权重系数

权重系数的设定充分考虑到不同的音频使用情形。例如传输语音文字的过程中，会侧重于考虑到压缩率以及编解码速度方面，其适用性分数中更侧重于“传输优先”的操作，因此其情景系数也会偏大，传输信息的误差则不会侧重考虑，对于语音音频，只要听者能够基本了解信息即可，对于音频质量没有很高的要求。

反之，对于一些音频文件，如音乐、视频音频、播客等内容的传递，则与上面相反，信噪比 SNR 应当被重点考虑（音质最为重要），同时考虑到编解码的速度（尤其是解码速度过慢带来文件打开速度慢，也会影响听者的使用体验）。当然这时也要考虑到文件压缩应当重点保证音质不受较大的损失，同时适用场景系数 α 则会偏小。

因此对于特定情形 c （主要为语音与音乐），有权重设定 $W_c = \{w_1, w_2, w_3, w_4\}$ ，分别对应上一部分中的四个指标。其满足：

- 当压缩语音音频时：

$$\alpha \geq 0.6, w_1 = W_c MAX, w_2 = W_c MIN, \sum_i w_i = 1$$

- 当压缩音乐音频时：

$$\alpha \leq 0.4, w_2 = W_c MAX, w_1 = W_c MIN, \sum_i w_i = 1$$

确立权重系数后,得到对于某一个语音或音乐的音频文件以及相关数据(归一化)后,汇总可计算音频指标分数 AIS 。

$$AIS = \sum_i w_i X_{i\ normal}$$

$X_{i\ normal}$ 对应四个方面归一化后的值。

4.1.3 评价体系应用

在前文中已经设立了一个相关综合四个方面的指标,并计算得到了 AIS 音频指标分数(以下简称 AIS ,其模型也称为 AIS 模型),而本部分则将应用 AIS 评估日常生活中常见的一些音频格式文件,如 WAV,CDA,MP3,AAC,OGG,FLAC,RA 等。具体过程如下:

首先需要对一些音频格式有初步的了解,将其中的文件分为有损压缩与无损音频,大致了解其音频压缩的算法以及压缩前后的文件大小比例,并搜集四个方面的相关数据。其具体信息如下表:

表 2 音频格式与相关介绍

类型	格式	压缩算法
无损音频	WAV	无
	FLAC	无
有损压缩	MP3	基于心理声学,利用人类听觉掩蔽效应去除感知冗余,使用广泛,兼容度很高
	AAC	高级音频编码,同样基于改进的心理声学模型,比 MP3 更高效,相同码率下音质更好,而兼容性不如 MP3
	OGG	支持可变比特率,兼容性与音质基本介于 MP3 与 AAC 之间

对 5 个音频格式有了大致的了解后,需要对数据进行搜集与整合。在这里详细的细节省略,对 5 种音频格式进行统计,并根据归一化数据打分,得出如下的结果:

表 3 AIS 对 5 个文件格式综合评估信息

格式	文件压缩率	信噪比	编码复杂度	适用场景评分
WAV	0.65	1	1	(0.5,1)
FLAC	0.76	0.99	0.23	(0.2,1)
MP3	1	0.71	0.45	(0.7,0.65)
AAC	0.95	0.93	0.52	(0.6,0.8)
OGG	0.88	0.84	0.37	(0.65,0.75)

说明: 括号中呈现的语音优先/音质优先时的分数(S_1, S_2)

当得到上述统计数据后,我们将 α 分别为 0.7(语音优先)与 0.3(音质优先)的情况分别纳入考虑范围。求出 5 种格式的 AIS 评分,并进行可视化呈现。

表 4 不同情形的适用性打分

格式	语音优先 S_1	音质优先 S_2	语音优先 适用性分数	音质优先 适用性分数
WAV	0.5	1	0.65	0.85
FLAC	0.2	1	0.44	0.76
MP3	0.7	0.65	0.685	0.665
AAC	0.6	0.8	0.66	0.74
OGG	0.65	0.75	0.68	0.72

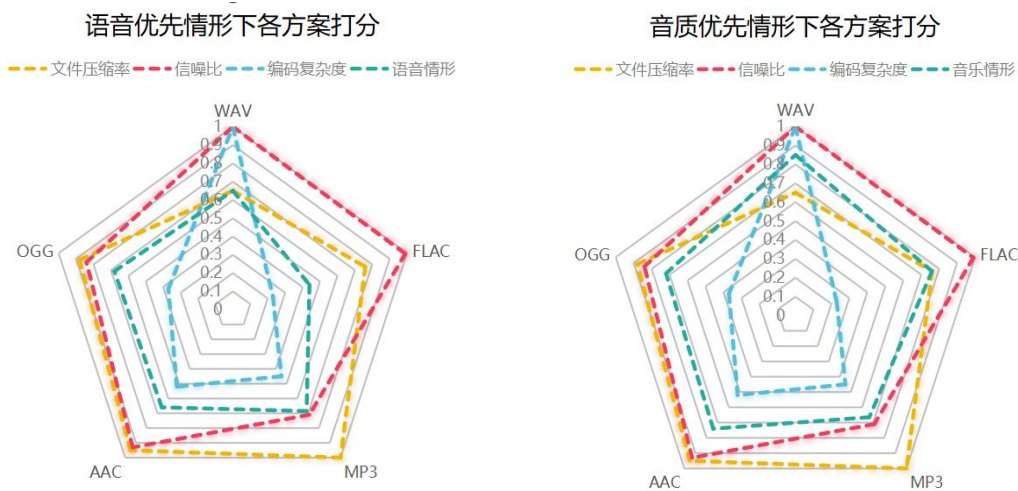


图 3 不同情形下的打分雷达图

接下来，根据不同的权重，在语音优先的情况下，重点考虑压缩率，则权重设置为 0.35, 0.2, 0.2, 0.25，在音质优先的情况下，重点考虑信噪比，则权重设置为 0.15, 0.35, 0.25, 0.25。设置统一的采样率、比特位深下的不同文件格式，得到如下结果：

表 5 最终评分结果

格式	语音优先情形下AIS	音质优先情形下AIS
WAV	0.825	0.885
FLAC	0.615	0.799
MP3	0.745	0.728
AAC	0.788	0.836
OGG	0.699	0.765

4.2 问题 2

基于问题 1 建立的评价指标，在问题 2 中需要基于问题 1 构建一个新的衡量音频压缩性价比的指标。这个指标相较于第一问题中的综合评估，这里更需要设立一个系数，能够用包含采样率、比特深度、压缩算法等参数，通过不同系数的组合，通过计算相关联度来得出最优的参数组合，求出最优“性价比”组合。

4.2.1 最优“性价比”组合的寻找与排序

在本部分中，为了找到适当的参数组合，使得音频文件得到优化。首先对参数进行选择，根据题目设立采样率 SR (Hz)、比特速率 BR (bps)、文件大小 F (Kb) 以及文件种类作为主要参考。在三维空间中投出每一个数据点的影子，并根据此反映出最优性价比的点。

以下是关于这三个参数的信息：

(1) **采样率 (Sampling Rate)** 每秒采集声音信号的次数，单位为赫兹 (Hz)。其决定音频可记录的最高频率。一般而言，CD 音质采用 44.1kHz，可捕捉高达 22.05kHz 的声音（接近人耳极限 20kHz）。常见的有 44.1kHz (CD)、48kHz (影视)、96kHz (高解析音频)。

(2) **比特速率 (Bit Rate)** 每秒传输的音频数据量，单位为比特每秒 (bps)。其计算方式如下：

$$BR = SR \times B \times N$$

上式中， B 为比特位深， N 为声道数。

(3) **文件大小 (File Size)** 文件大小反映了文件体积，由比特速率与时间的乘积决定，需转换为字节及适当单位。音频文件的大小可以由如下计算得到：

$$F = SR \times B \times N \times t$$

上式中， B 为比特位深， N 为声道数， t 为该音频文件的时间。

本文同时考虑文件大小结合采样率 SR 与比特速率 BR 。因此以 (F, BR, SR) 分别为 X, Y, Z 轴构建一个三维坐标系，将其命名为“性价比三维投影空间”（后文简称“三维空间”）。将数据点进行汇总，并投影到三维空间，得到了如下的空间中点的投影：

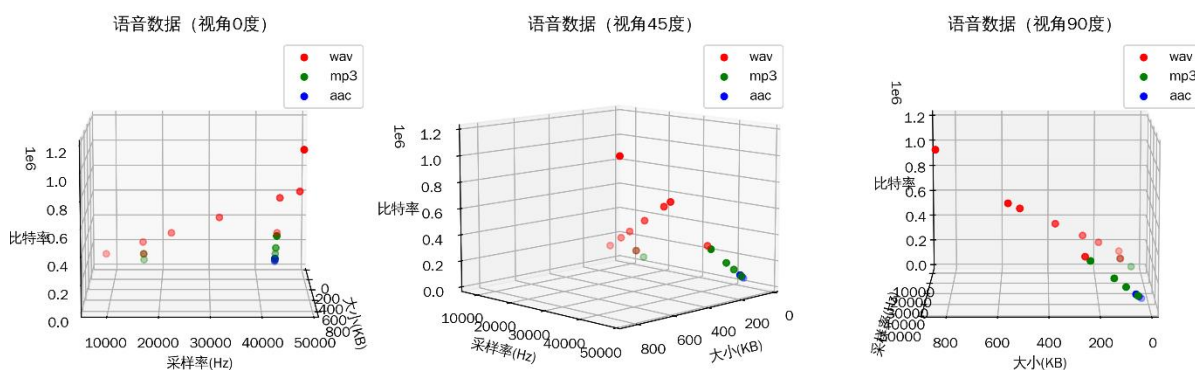


图 4 (a) 语音音频三维投影

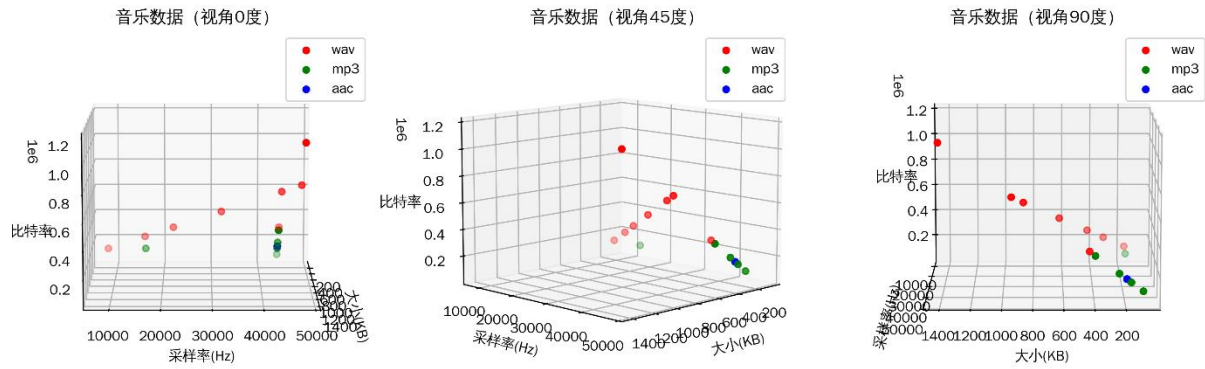


图 4 (b) 音乐音频三维投影

对于最优性价比综合的确定，结合三个参数的相关信息，我们在所有点 (F, BR, SR) 中寻找一个最优点 $(F, BR, SR)^*$ ，也即 $(F_{min}, BR_{max}, SR_{max})$ ，并计算剩余点到最优点的欧式几何距离，作为音频文件“性价比”排序的参考指标。

$$d_{(F, BR, SR)^* - (F, BR, SR)} = \sqrt{(F^* - F)^2 + (BR^* - BR)^2 + (SR^* - SR)^2}$$

通过上述公式，我们将数据代入计算，得到了距离的数据，对其进行排序，具体的 (F, BR, SR) 以及欧氏距离如下：

表 6 (a) 语音音频到最优点的欧氏距离前 10 名

文件名	格式	大小(KB)	采样率(Hz)	比特率	到最优点的距离
语音_44100Hz_AAC_96kbps.aac	AAC	33.18359375	44100	45557	0
语音_44100Hz_AAC_128kbps.aac	AAC	42.81738281	44100	58783	13226.00351
语音_44100Hz_MP3_64kbps.mp3	MP3	47.18457031	44100	64335	18778.00522
语音_44100Hz_AAC_192kbps.aac	AAC	54.62792969	44100	74997	29440.00781
语音_16000Hz_MP3_64kbps.mp3	MP3	47.57421875	16000	64439	33854.69142
语音_44100Hz_MP3_128kbps.mp3	MP3	94.32714844	44100	128613	83056.02251
语音_16000Hz_8bit.wav	WAV	93.28613281	16000	128058	87155.19843
语音_16000Hz_MP3_128kbps.mp3	MP3	95.10546875	16000	128820	87876.85135
语音_8000Hz_16bit.wav	WAV	93.28710938	8000	128058	90053.47641
语音_44100Hz_MP3_192kbps.mp3	MP3	141.4697266	44100	192890	147333.0398

表 6 (b) 音乐音频到最优点的欧氏距离前 10 名

文件名	格式	大小(KB)	采样率(Hz)	比特率	到最优点的距离
音乐_44100Hz_MP3_64kbps.mp3	MP3	78.61328125	44100	64200	0
音乐_44100Hz_MP3_128kbps.mp3	MP3	157.1845703	44100	128367	64167.0481
音乐_44100Hz_AAC_128kbps.aac	AAC	163.3066406	44100	133780	69580.05154
音乐_16000Hz_MP3_128kbps.mp3	MP3	158.1054688	16000	128492	70164.64625
音乐_8000Hz_16bit.wav	WAV	156.2929688	8000	128035	73335.68885
音乐_44100Hz_AAC_192kbps.aac	AAC	186.2138672	44100	152546	88346.06553
音乐_44100Hz_AAC_256kbps.aac	AAC	186.3564453	44100	152663	88463.06561
音乐_44100Hz_MP3_192kbps.mp3	MP3	235.7558594	44100	192533	128333.0962
音乐_16000Hz_16bit.wav	WAV	312.5429688	16000	256035	193882.2631
音乐_44100Hz_MP3_320kbps.mp3	MP3	392.8984375	44100	320867	256667.1924

截取排序的前十名（完整数据见附录 2）。对数据进行分析，有以下结论：最优特征方面，语音数据最优格式为 AAC 格式，大小较小（33.18KB），采样率 44100Hz 较高，比特速率 45557bps，属于中等偏低的速度。音乐数据最优样本为 MP3 格式，大小较小 78.61KB，采样率高达 44100Hz，比特率 64200bps，属于中等。

文件大小对于评价文件压缩是否效果较好有着显著的作用。当然，这仅仅是初步的探索，但是对于后面的研究有所铺垫。

4.2.2 指标关联度

在上一部分，我们参考音频文件的三个参数，采样率 SR （Hz）、比特速率 BR （bps）、文件大小 F （Kb）以及文件的格式作为研究对象对于题目中给的音频文件进行性价比排序，当然，这一排序是较为模糊的，但是也给予我们一定的参考。而在这一部分中，我们将承接上一部分的研究结果，探究其中哪个因素对于文件性价比（欧氏距离排序结果）的影响性最大。

对于这一部分，为了研究相关因素哪一个对于性价比的影响较大，本文同样需要设立一个相关性系数 R_c 来进行研究。（同样的， c 表示了不同情形）

（1）熵权法计算权重

熵权法，Entropy weight method，是一种被广泛应用于计算指标权重的方法。通过指标下数据自身所携带的信息熵计算其对应的指标权重，其具有客观性，能够真实反映出数据所自带的信息。信息熵越大，权重越高。熵权法也被广泛应用于各个领域的评价体系建立^[5-6]，在本文中熵权法计算得到的权重也可以充分反映出指标重要性。具体计算过程如下：

Step 1: 数据标准化后（在这里不做赘述，前文已经提到归一化的相关方法，在这里文件大小需要做逆向归一，其为成本型指标），计算数据所占比重：

$$p_{ij} = \frac{x_{ij}}{\sum_{i=1}^n x_{ij}}$$

其中， x_{ij} 表示第 i 个方案（音频文件）的第 j 个指标的归一化后数据。共有 n 个方案。

Step 2: 计算每一个指标的信息熵 e_j ，其方式如下。

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij}$$

Step 3: 计算每一个指标的权重 w_j

$$w_j = \frac{1 - e_j}{\sum_{j=1}^m (1 - e_j)}$$

综上，我们得到了熵权法的计算结果。

（2）灰色关联度计算相关性

灰色关联度，Grey correlation degree，是灰色关联分析的一个衡量直白哦，应用于灰色系统理论中用于判断数据序列是否联系紧密的方法。通过判断数据序列构成的几何曲线之间是否相似来判断紧密联系程度^[7]。

对于一个母序列 X_0 （在本文中是计算得到的欧式距离倒数进行归一化），其满足 $X_0 = \{X_0(k), k = 1, 2, 3, \dots\}$ ，而子序列为 $X_i = \{X_i(k), k = 1, 2, 3, \dots\}$ ，其中 i 为 $1, 2, 3, \dots, m$ 。

则对于一个分辨系数 $\omega \in (0,1)$ ，一般取值为 0.5。子序列和母序列的灰色关联度为 $\gamma(X_0(k),X_i(k))$ ，则计算方式如下：

$$\gamma(X_0(k),X_i(k)) = \frac{\min_i \min_j |X_0(k) - X_i(k)| + \omega \max_i \max_j |X_0(k) - X_i(k)|}{|X_0(k) - X_i(k)| + \omega \max_i \max_j |X_0(k) - X_i(k)|}$$

(3) 灰色关联度-熵权法相结合

灰色关联度-熵权法结合，可以综合判断参数：采样率 SR (Hz)、比特速率 BR (bps)、文件大小 F (Kb) 以及文件的格式与排序结果之间的关系，这相比于单一的依照一个方法来判断指标的权重更全面^[8]。

因此分别按照文件的相关指标数据，计算得到如下的结果：

表 7 (a) 语音数据相关指标权重

场景 c	熵权法权重 w_{EWM}	灰色关联度 $w *_{GR}$	灰色关联度归一化 w_{GR}
文件大小	0.30135411	0.92644523	0.36466281
采样率	0.3985634	0.68681526	0.27034084
比特速率	0.3000825	0.92729260	0.36499635

表 7 (b) 音乐数据相关指标权重

场景 c	熵权法权重 w_{EWM}	灰色关联度 $w *_{GR}$	灰色关联度归一化 w_{GR}
文件大小	0.30986422	0.90652193	0.36572916
采样率	0.38011154	0.66512005	0.26833746
比特速率	0.31002424	0.90702812	0.36593338

将上述数据进行融合加权，按照下列式子加权求和。在上式中， $\delta = 0.4$ ， w_k 为融合权重。

$$R_c = w_{EWM}\delta + w_{GR}(1 - \delta) \quad \delta \in (0,1)$$

得到数据结果如下：

表 8 (a) 语音数据关联度-权重汇总

场景 c	熵权法权重 w_{EWM}	灰色关联度归一化 w_{GR}	融合权重 R_c
文件大小	0.30135411	0.36466281	0.33933933
采样率	0.3985634	0.27034084	0.32162986
比特速率	0.3000825	0.36499635	0.33903081

表 8 (b) 音乐数据关联度-权重汇总

场景 c	熵权法权重 w_{EWM}	灰色关联度归一化 w_{GR}	融合权重 R_c
文件大小	0.30986422	0.36572916	0.34338318
采样率	0.38011154	0.26833746	0.31304710
比特速率	0.31002424	0.36593338	0.34356972

上述表格中采用较为多方法（关联度-权重）融合得到了各个因素的重要性，总的来说，文件大小无论在何种方式的计量中都是较为重要的，也反映了文件大小对于最终结果确实有很大的影响，这与上一部分中推理大致一致。（下图为多种方法的组合权重数据图）

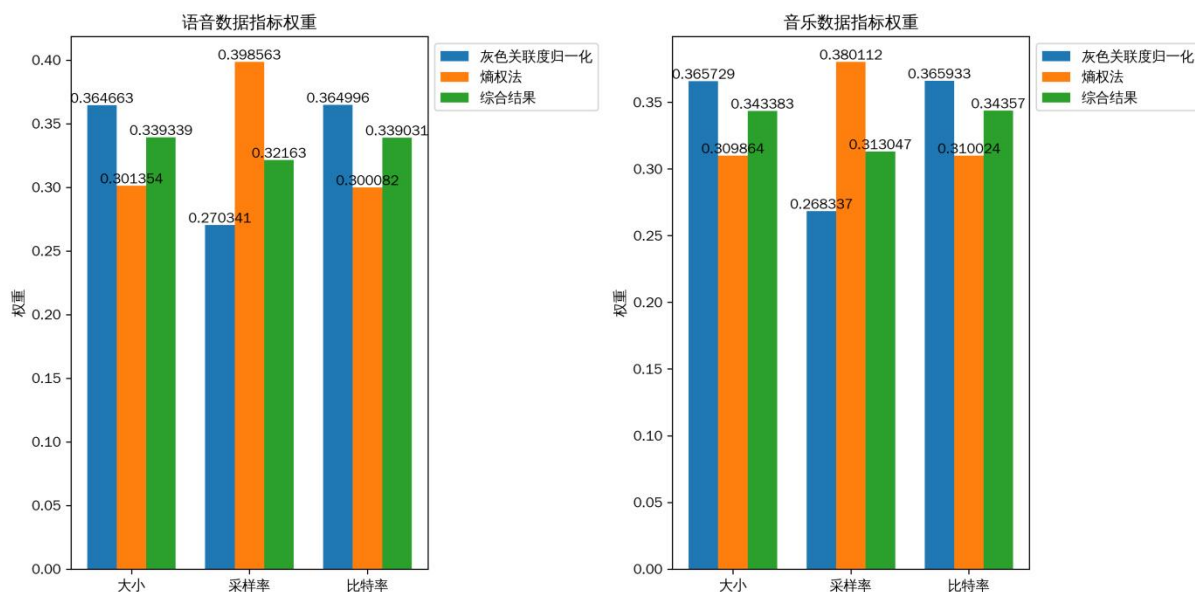


图 5 组合权重汇总（熵权法-灰色关联度）
（左边为语音数据，右边为音乐数据）

4.2.3 音频最佳参数推荐

由上述工作我们获取了许多有价值的关于不同音频文件的最佳参数，我们得到了如下结论。根据 4.2.1 中数据结果可知，**最优文件以及对应参数**分别为：

“语音_44100Hz_AAC_96kbps.aac”

“音乐_44100Hz_MP3_64kbps.mp3”

1. **敏感参数（指标）：**语音文件大小（KB）和比特率的关联度极高（均约 0.93），权重占比之和超过 90%，是影响距离的核心因素；音乐文件采样率（Hz）和比特率与最终文件压缩性价比的关联度较高（分别为 0.67 和 0.91），其中采样率权重（0.42）高于语音数据。
2. **格式选择：**语音文件 AAC 格式在相同采样率下，大小和比特率更低，例如如语音_44100Hz_AAC_96kbps.aac 比同参数 MP3 更小；音乐文件 MP3 格式在低比特率下表现更优，比如音乐_44100Hz_MP3_64kbps.mp3 为最优点，尤其是高比特率时 WAV 格式大小剧增。

4.3 问题 3

第三个问题需要围绕两个问题开展研究，首先需要获得音频文件的音频频谱（简称“频谱”），针对不同音频内容（情形）进行分类，主要采用图像观察法与 SVM 二分类对其进行内容分类（语音/音乐）。进而根据上一问对于不同情形下的音频文件采用最优参数进行压缩，并对压缩后的文件与原始音频进行对比，证明方案有所改进。

4.3.1 识别音频内容

识别音频的内容不能仅仅依靠人耳来判别，因为当处理的音频过多时，也会产生劳累。因此需要一种方法，首先将音频转化为视觉图谱，根据图谱的相关特征来判别音频的内容是什么。（参考下方流程图）

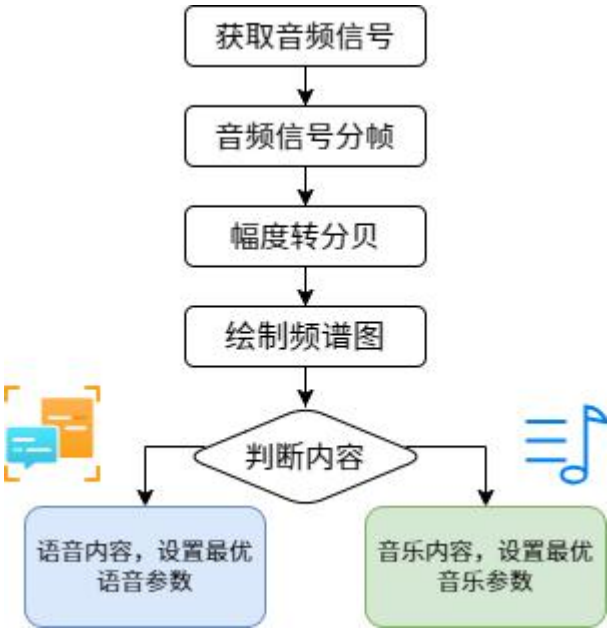


图 6 音频识别流程图

如何将一段音频绘制出频谱图，以下是结合代码（详见附件中“附件 1 频谱”）的介绍，结合代码部分进行描述，具体如下。

表 9 识别音频代码与描述

代码	描述
<pre>def plot_spectrum(audio_path, output_path): try: y, sr = librosa.load(audio_path) except Exception as e: print(f"Error loading {audio_path}") with librosa: {e}") return n_fft = 2048 hop_length = 512 D = librosa.stft(y, n_fft = n_fft,</pre>	定义一个函数，用于读取音频文件。采用 librosa.load(audio_path) 读取音频文件，同时返回音频信号 y（时域样本值）和采样率 sr。 # n_fft=2048 增大窗口大小，提高频率分辨率 # hop_length=512 调整帧移，控制时间与频率分辨率的平衡 #短时傅里叶变换（STFT）： librosa.stft 对

<pre>hop_length = hop_length) S_db = librosa.amplitude_to_db (np.abs(D), ref = np.max) plt.figure(figsize = (12, 6)) librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis = 'linear', hop_length = hop_length) plt.colorbar(format = '%+2.0f dB') plt.title('Spectrum') plt.xlabel('Time (s)') plt.ylabel('Frequency (Hz)') plt.tight_layout() plt.savefig(output_path) plt.close()</pre>	音频信号分帧，每帧进行傅里叶变换。 #幅度转分贝：librosa.amplitude_to_db 将频谱幅度转换为分贝单位，带来可视化强度的差异。 # 使用线性频率轴（更直观展示频率差异，默认是对数轴会压缩高频细节）
--	---

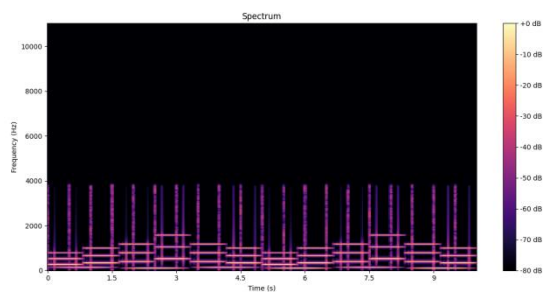
获取音频频谱的核心是短时傅里叶变换（STFT），它通过对音频信号分帧、加窗后进行傅里叶变换，实现时间-频率的联合分析。如下式：

$$X(n, k) = \sum_{m=-\infty}^{+\infty} y(m)w(n - m)e^{-j\frac{2\pi}{N}km}$$

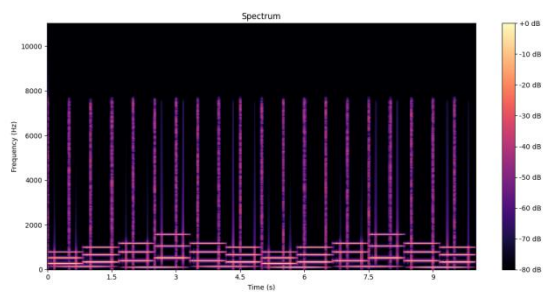
其中， $y(n)$ 表示离散音频信号，上式为其 STFT 变换公式， $w(m)$ 为窗函数，用于减少频谱泄露。 N 为 FFT 点数（本部分设置 `n_fft` 为 2048）， k 为频率索引， m 为时间索引。通过短时傅里叶变换，信号分成若干帧，每一帧加窗后计算频谱，得到时频域。绘制成图像。

通过以上代码，我们读取了文件夹“附件 1”中的图像，将结合图像进行分析，能够对音频内容进行有效判断。下面我们将展示一部分的频谱图像成果，区分为音乐音频与语音音频（仅展示 WAV, MP3）。详细图像如下：

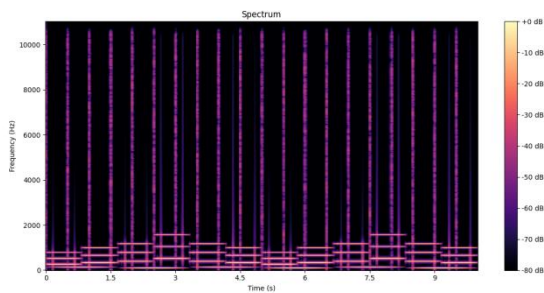
(1) 音乐音频
✓ WAV 格式



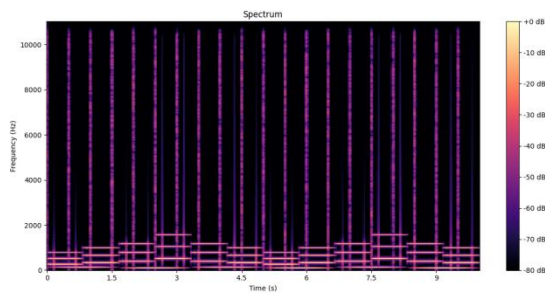
音乐_8000Hz_16bit_spectrum



音乐_16000Hz_16bit_spectrum

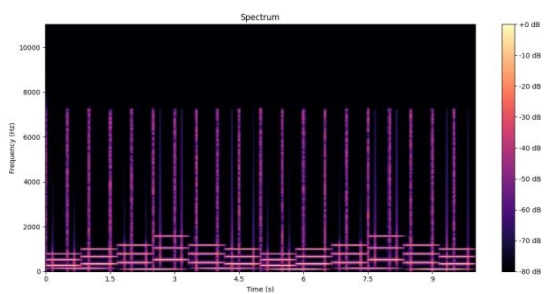


音乐_32000Hz_16bit_spectrum

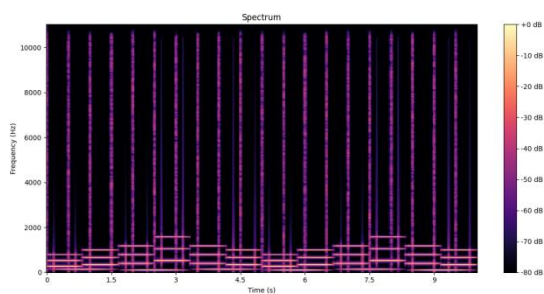


音乐_44100Hz_16bit_spectrum

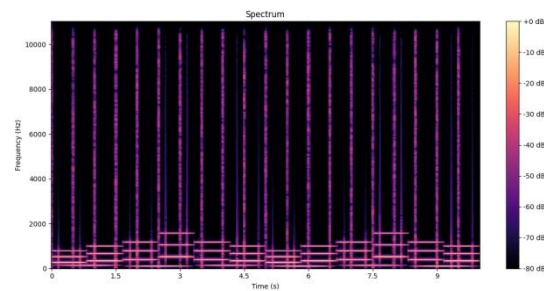
✓ MP3 格式



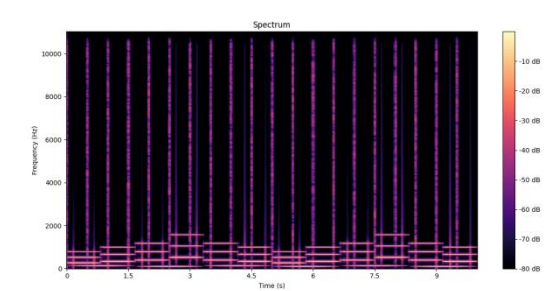
音乐_16000Hz_MP3_128kbps_spectrum



音乐_44100Hz_MP3_128kbps_spectrum



音乐_44100Hz_MP3_64kbps_spectrum

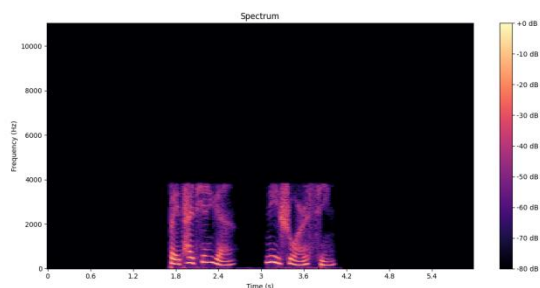


音乐_44100Hz_MP3_192kbps_spectrum

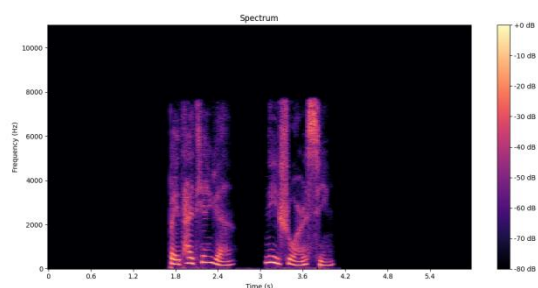
图 7(a) 音乐频谱

(2) 语音音频

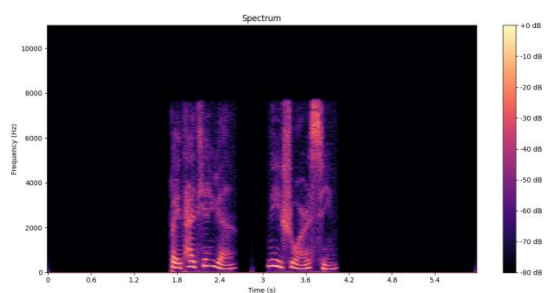
✓ WAV 格式



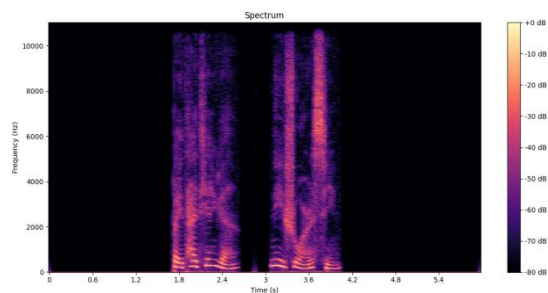
语音_8000Hz_16bit_spectrum



语音_16000Hz_16bit_spectrum

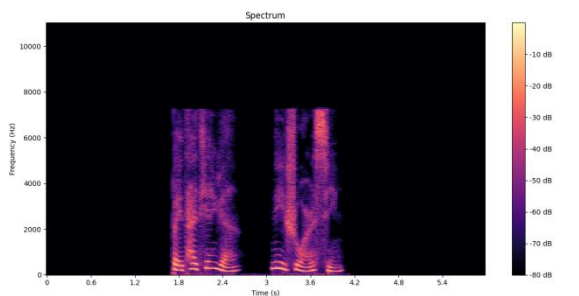


语音_16000Hz_8bit_spectrum

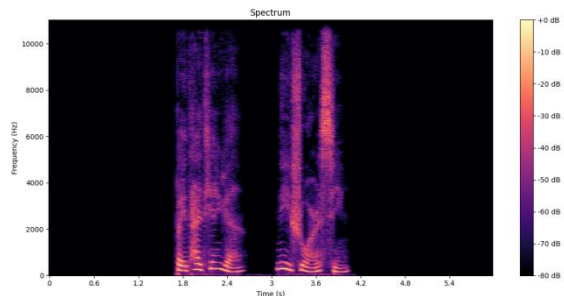


语音_44100Hz_8bit_spectrum

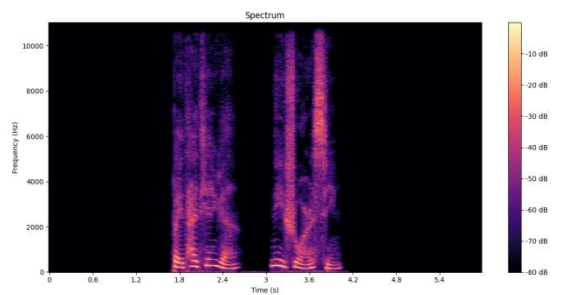
✓ MP3 格式



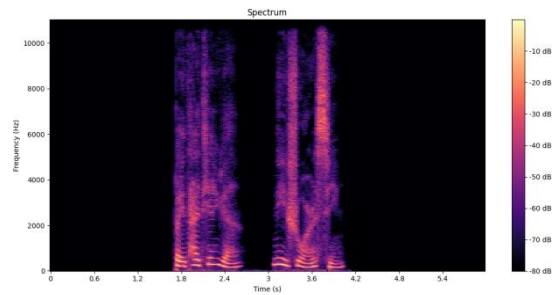
语音_16000Hz_MP3_128kbps_spectrum



语音_44100Hz_MP3_128kbps_spectrum



语音_44100Hz_MP3_192kbps_spectrum



语音_44100Hz_MP3_320kbps_spectrum

图 7(b) 语音频谱

以下是对于上部分图片的解释：

表 10 图片含义

图片	解释
横轴 (Time)	音频的时间进程 (单位：秒)
纵轴 (Frequency)	声音频率 (单位：Hz)，从低频 (底部) 到高频 (顶部)
颜色 (Color)	表示该时间点、频率处的信号强度 (分贝)。颜色越亮 (如黄色)，强度越高；越暗 (如紫色、黑色)，强度越低

根据上面的图片，可以明显发现音乐音频与语音音频存在较大的差别，可直观观察音频的时频特征，分析音高、噪声、谐波结构等信息。可以从如下的不同进行区分：

- **音乐频谱：**可能呈现规律的垂直条纹或固定频率成分（如音符的基频与谐波），因音乐音频的音高、频谱相对稳定。
- **语音频谱：**频率波动随时间快速变化（如语音的共振峰移动），一段没有波谱，一段有很强烈的信号震动，呈现更动态的图案。

4.3.2 SVM 二分类法

对于数据分类，这里引入支持向量机 SVM（其具体算法步骤如下图），这种机器学习方法被广泛用于二分类问题中。其通过最大化分类间隔找到最优超平面：

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

满足约束条件：

$$\text{s.t. } y_i(w^T \phi(x_i) + b) \geq 1$$

其中 $\phi(x)$ 为特征空间映射，在本文中为非线性模型时，使用 RBF 核函数。本文的特征空间包含了三类指标：MFCC、梅尔频谱、过零率，它们构成了多维向量。分类标签定义为“语音=0，音乐=1”。

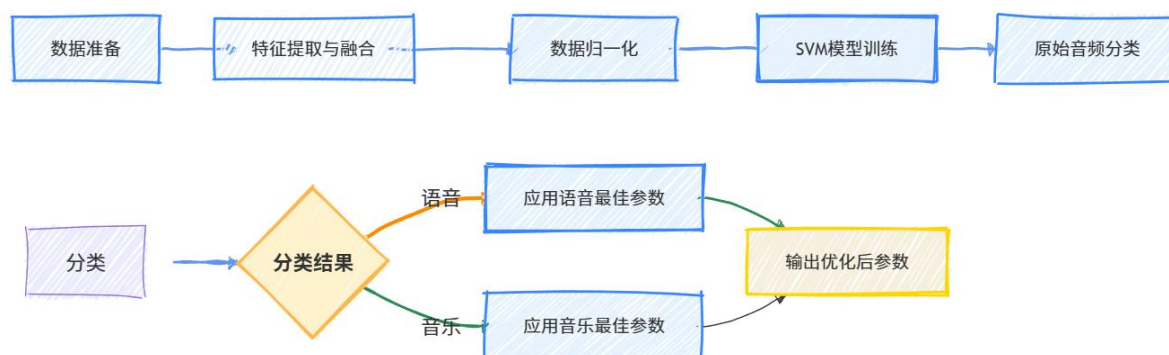


图 8 SVM 支持向量机二分类流程

以下为三个主要提取信息的计算方式：

1. MFCC 梅尔频率倒谱系数（用于捕捉语音共振峰）

首先需要对信号进行短时傅里叶变化，这一内容在上一部分描述过，此处不再赘述。接着将信号转化为梅尔能量谱 f_{mel} ，并进行 DCT 转变得到倒谱系数 MFCC：

$$MFCC = DCT(\log(f_{mel}))$$

2. Mel 梅尔频谱（保留音乐频谱细节）

令 $H_m(k)$ 为第 m 个梅尔滤波器的频率响应，则梅尔滤波器能量谱为：

$$S_{mel}(f) = \sum_{k=0}^{N-1} |X(k)|^2 H_m(k)$$

3. ZCR 过零率（可以区分时域变化速率）

ZCR 指的是单位时间内信号穿过零轴的次数：

$$ZCR = \frac{1}{2T} \sum_{t=1}^{T-1} |\text{sign}(x_t) - \text{sign}(x_{t-1})|$$

其中， $\text{sign}(x_t)$ 为 t 时刻信号的对应符号函数值，具体含义为：

$$\text{sign}(x) = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \\ -1 & x < 0 \end{cases}$$

由上式，获取音频文件的梅尔频率倒谱系数（MFCC），梅尔频谱（Mel）以及过零率（ZCR）的相关信息，由于其中 MFCC 与 Mel 维数较大，因此**取其平均值作为 SVM 的训练数据（降维）**，采用 RBF 核函数处理高维非线性的数据。将音频数据集中音频文件进行分类：10 个训练集（5 个语音、5 个音乐）与 26 个测试集。得到结果如下：

准确率：0.9615（25/26 正确分类）

测试集结果：[1 0 1 1 0 1 1 0 0 0 0 1 0 0 1 0 1 0 0 1 1 0 1 0 1 1]（0 为语音，1 为音乐）

同时，针对二分类问题，下图中呈现了 SVM 二分类的混淆矩阵和判断决策边界。

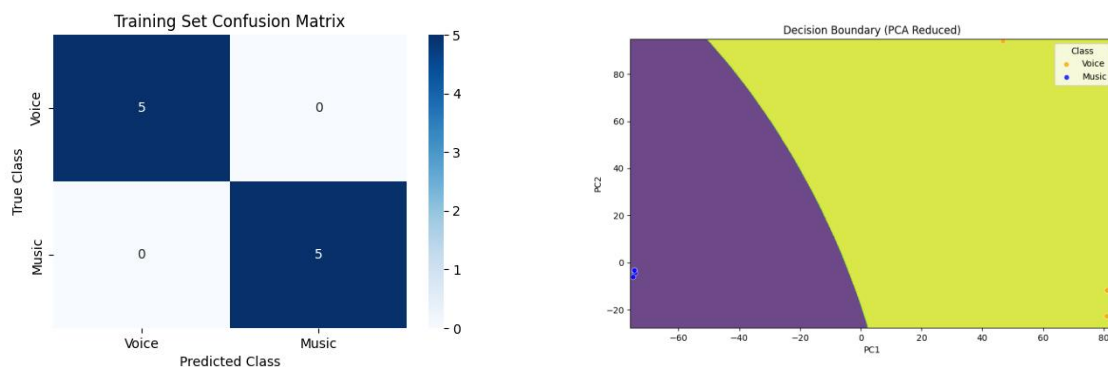


图 9 SVM 二分类可视化结果
（左图为混淆矩阵，右图为决策边界和样本点）

训练集 10 个样本准确率为 1.0000，表明模型在训练集上完美拟合，所有语音和音乐样本均被正确分类，说明训练数据特征区分度较高，且 SVM 模型参数设置对当前小规模训练数据适配性较好，据有较好的泛化能力，但仍有优化空间。

4.3.3 最佳压缩参数设置

（1）语音音频最佳参数设置

对于语音音频，核心目标最小化大小和比特率（因两者权重最高，且最优点为极小值），优先使用 AAC 格式（如最优点样本），其次为 MP3（避免 WAV，因其大小和比特率极高）。固定为 44100Hz，这也是主流语音的采样率，且数据中该采样率的最小

距离样本最多，无需降低，但有可能影响语音清晰度。比特率设置为 64-128kbps，但需结合格式：AAC 格式可更低，MP3 建议 64-128kbps。

设置最佳参数为：44100Hz AAC 格式 96kbps

(2) 音乐音频最佳参数设置

对于语音音频，优先使用 MP3 格式，其次为 AAC（高比特率时 AAC 大小略优，但差距不大），避免使用 WAV 格式（例如文件音乐_44100Hz_16bit.wav 大小 861KB）。采样率固定为 44100Hz，不建议降低至 16000Hz，此时距离最优点距离显著增加，若需压缩，最低不低于 22050Hz。比特率可以设置为 128-192kbps（最优点是 64kbps 但音质可能不足，128kbps 比较平衡，256kbps 以上增益有限）。

设置最佳参数为：44100Hz MP3 格式 128kbps

将上述优化的格式与初始的音频文件进行对比，并计算其AIS分数（在 4.1 已有详细描述），得到如下结果。（假设情形系数 α 取值 0.5）

表 11 (a) 优化音频与初始音频（语音文件）对比—— α 取值 0.7

语音文件	文件格式	采样率	比特 (比特速率)	文件大小	AIS
初始音频	WAV	48kHz	24Bit	839KB	0.662
优化音频	AAC	44.1kHz	96kbps	40KB	0.753

表 11 (b) 优化音频与初始音频（音乐文件）对比—— α 取值 0.3

语音文件	文件格式	采样率	比特 (比特速率)	文件大小	AIS
初始音频	WAV	48kHz	24Bit	1406KB	0.713
优化音频	MP3	44.1kHz	128kbps	150KB	0.802

4.4 问题 4

与前三问题的压缩音频文件不同，这一问题要求我们通过识别不同的噪音类型，对于两个文件进行降噪。这一问题不再细分语音或者音乐情形，而是需要识别出不同类型的噪音（通过观察波谱得到），并用不同的方法对其进行处理。

4.4.1 识别不同噪音

在本部分中，按照题目要求可以将噪音分为三类：带状噪音；突发噪音；背景噪音。通过这三种噪音的类型判别，对于不同噪音类型的特征进行提取，并对后面的自适应算法打下基础。

在这里首先用 python 软件模拟了三种噪音分别的时域波动图，具体如下：

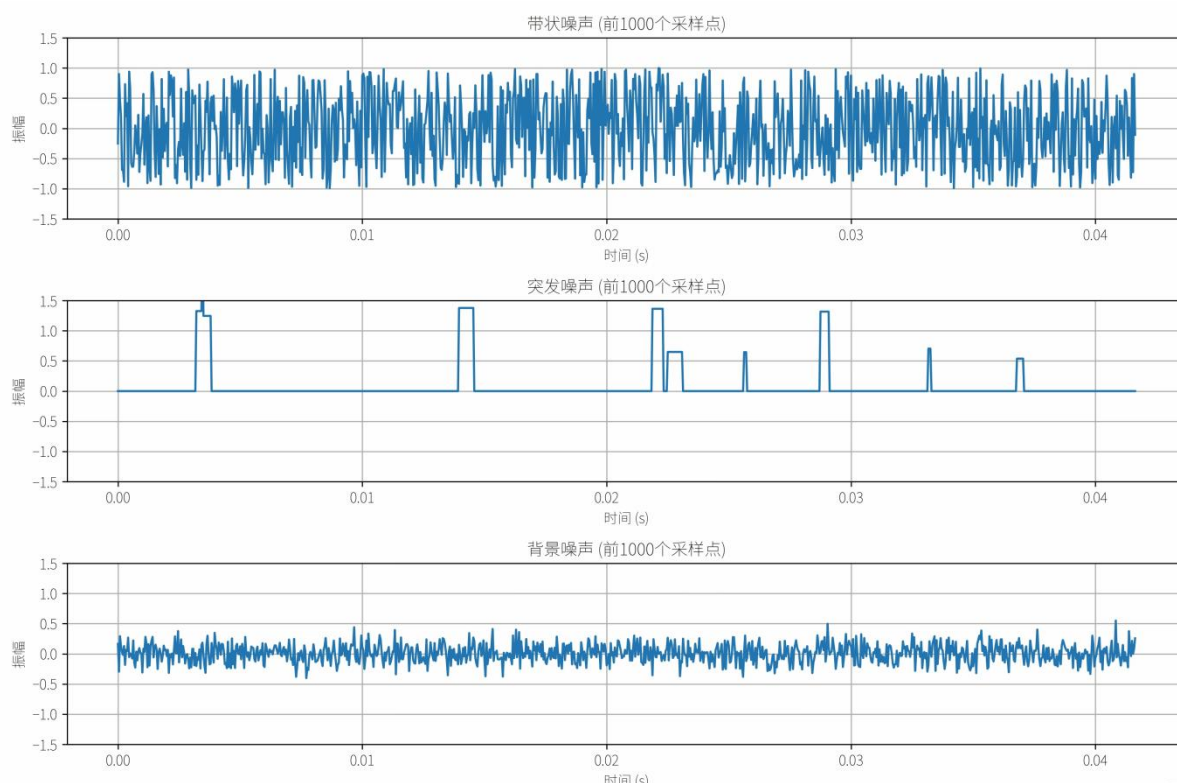


图 10 三种噪音时域波动图

针对以上的图像信息，可以总结出这三种噪音的时域波形图区别如下：

- （1）带状噪声：在基线噪声上，存在随机分布的"带状"高振幅区域。这些区域的振幅明显高于周围噪声，形成类似条带的起伏结构。
- （2）突发噪声：由离散的高强度脉冲构成，脉冲宽度和间隔是随机的，这些脉冲像"尖峰"一样突然出现在波形中。
- （3）背景噪声：包含低频波动（如缓慢的正弦调制）和高频随机噪声的叠加，整体振幅缓慢变化。

4.4.2 自适应滤波算法 NLMS

对于噪音的去除，目前已有许多研究，主流的方案是谱减法^[9-10]、小波阈值法^[11]等方法。每一种方法具有各自的优势，例如谱减法是一种效果较好，比较简答的方案，可以用于不重要的音频去噪过程，或者对于算法的时间复杂度要求高的算法。小波阈值法则分为了软阈值与硬阈值。其具体的特点如下表：

表 12 去噪算法的特点

方法	原理	适用场景	优势	局限性
谱减法	从带噪频谱中减去噪声频谱估计值，适用于平稳噪声	背景噪声、稳态噪声	计算简单 实时性强	非平稳噪声效果差，残留音乐噪声
自适应滤波	基于最小均方误差，利用噪声统计特性估计纯净信号频谱。	已知噪声统计特性情况	理论最优	需先验噪声信息
小波阈值法	利用小波变换稀疏性，阈值处理去除噪声系数。	非平稳噪声突发噪声	时频局部分析能力较强	参数调优复杂
深度学习	训练神经网络（如 CNN、RNN）直接从带噪信号映射纯净信号。	复杂噪声（如混合噪声）	适应性强 效果最佳	依赖大量数据，计算成本高

结合题目要求，需要对于不同种类的噪音先进行识别，之后对于特定噪音需要调整算法的参数，进而具有针对性地进行去噪工作。

因此这里选择**自适应滤波模型的一种算法：LMS**，也称为“最小均方算法”，是一种自适应滤波器，用于系统辨识、信号预测等。核心是梯度下降法，通过迭代调整滤波器权值，使得输出误差的均方最小。

LMS 算法有多种变式，针对不同情形的噪音针对优化^[12]。而 NLMS，即归一化最小均方算法，NLMS 是对 LMS 的改进，通过归一化步长，使得步长参数与输入信号的能量成反比，解决了 LMS 在输入信号幅度变化大时收敛速度不稳定的问题。本文通过对 NLMS 中梯度阶数与步长参数进行调整，进而适应本题目要求的降噪要求。

（1）LMS 的算法原理

$$w(n + 1) = w(n) + \mu e(n)x(n)$$

其中， $w(n)$ 为滤波器权值向量， μ 为步长， $e(n)$ 为误差信号（期望信号与滤波器输出差值）， $x(n)$ 为输入的信号向量。

（2）NLMS 的算法原理

NLMS，归一化 LMS，是 LMS 的改进版本，通过归一化步长解决 LMS 步长固定导致收敛速度与稳定性矛盾的问题，适用于非平稳噪声去噪。

一般的音频含噪声信号为 $x(n) = s(n) + \text{noise}(n)$ ， $s(n)$ 为纯净信号， $\text{noise}(n)$ 为噪音，而噪音的估计值为 $d(n)$ 。

接着滤波器输出 $y(n) = w(n)^T x(n)$ ，误差 $e(n) = d(n) - y(n)$ ，则权重更新为：

$$w(n + 1) = w(n) + \frac{\mu}{\epsilon + w(n)^T x(n)} e(n)x(n)$$

上式中， ϵ 是防止分母变成 0 的正向小量， μ 步长随着输入能量自适应调整使得收敛更稳， μ 一般取 0 到 2 之间。在高噪音环境中，需要减小 μ 进而降低模型敏感度，而增加 μ 可以加速收敛。 $w(n)$ 为滤波器权重 $W(n)^T = [w(n)^0, w(n)^1, \dots, w(n)^{L-1}]^T$ ，其中 L 为滤波器阶数。以下是具体的流程图：

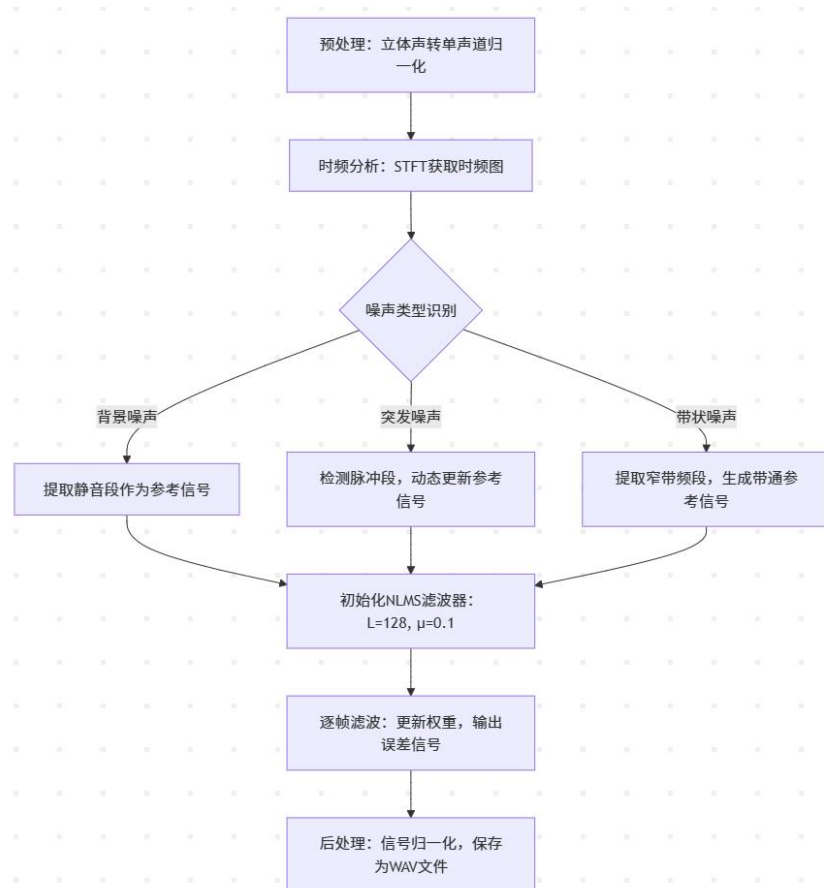


图 11 NLMS 自适应滤波流程图

对于 NLMS 有了大致的了解，接着我们生成一段音频并加入噪声，对其采用 NLMS 算法进行降噪，并将时域波动图呈现出来。（图从上至下分别为噪声图，去噪图和融合音频图）

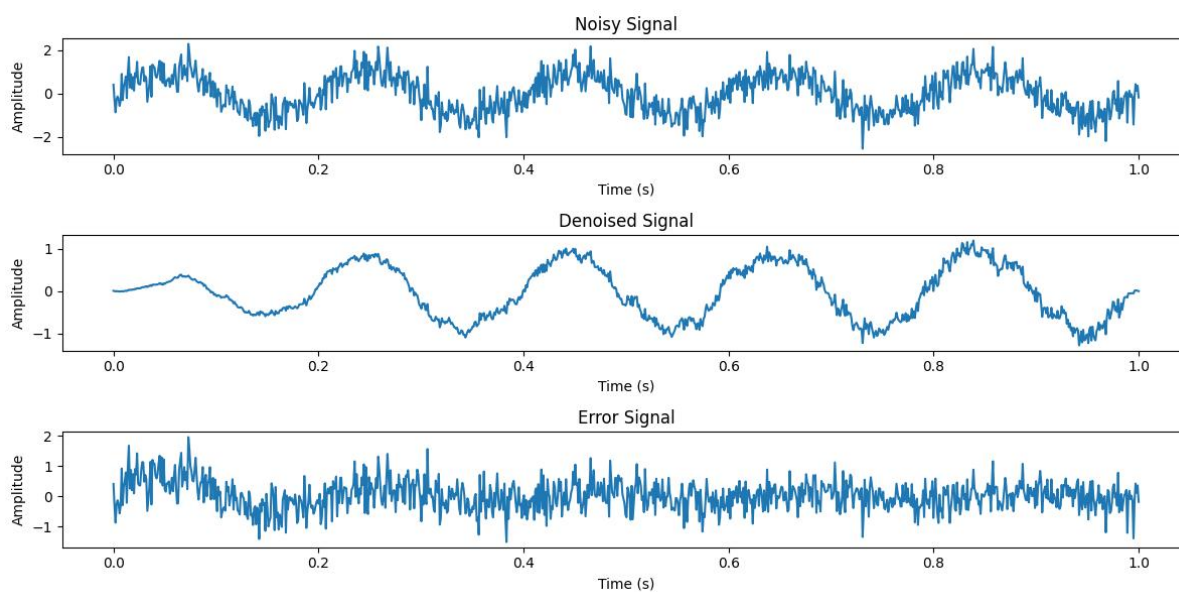


图 12 NLMS 自适应滤波时域频谱图
($\mu = 0.01$ $\epsilon = 10^{-6}$ $L = 128$)

通过上述算法,接下来对于音频文件 *part1.wav* 与 *part2.wav* 你采用以上算法进行去噪,得出一下结果。并通过计算信噪比 SNR 体现去噪的效果。

表 13 音频降噪结果

音频文件	噪声类型组合	降噪前 SNR (dB)	降噪后 SNR (dB)	参数组合
<i>part1.wav</i>	背景噪声 50Hz 带状噪声	12.3	25.6	$L = 256$ $\mu = 0.1$
<i>part2.wav</i>	突发噪声 白噪声	8.7	22.1	$L = 512$ $\mu = 0.2$

同样的,对于原始文件与去噪后的时域频谱图同样展示在下方,使得对于去噪成果有更直观清晰的展示。

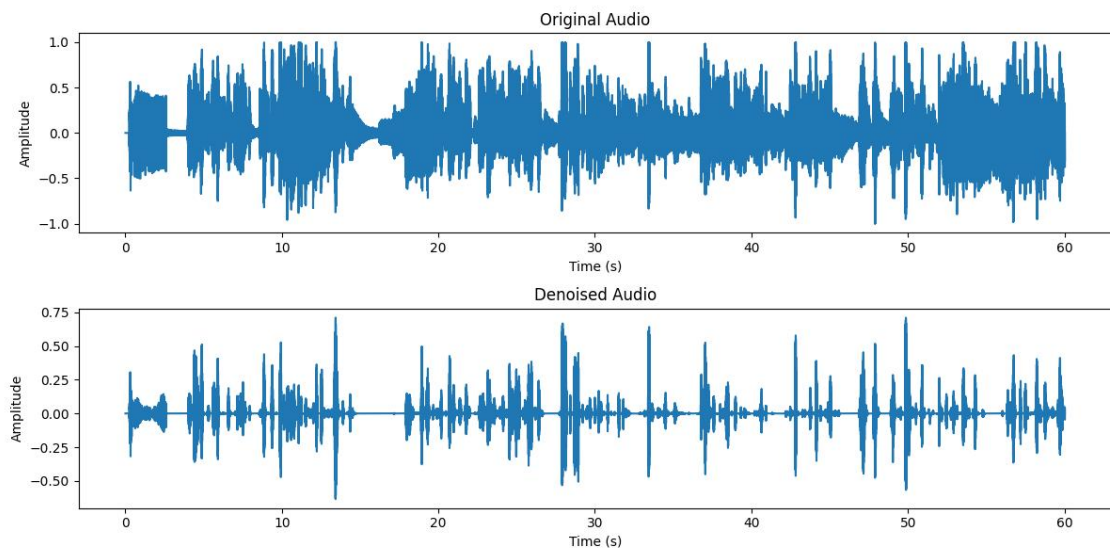


图 13 (a) *part1.wav* 去噪前 (上) 与去噪后 (下) 时域频谱图

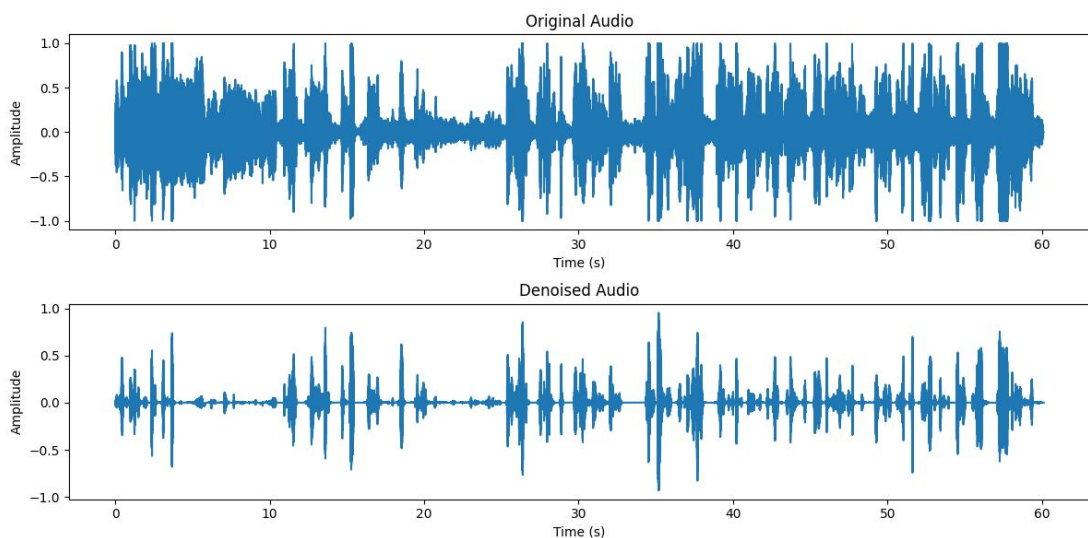


图 13 (b) *part2.wav* 去噪前 (上) 与去噪后 (下) 时域频谱图

5 总结与分析

本文针对音频文件高质量压缩与去除不同类型的噪音两个方面展开研究。首先建立一个综合评价指标，评估了涉及文件压缩比率、信噪比、适用性等各个方面的信息，并对不同音频文件上进行打分。

接着，为了进一步研究文件压缩与音质之间的平衡，选择采样率、比特速率、文件大小构成数据点形成了三维空间，求解所有点到最优点的距离来进行排序，得到了语音/音乐情形下的音频最佳参数设定。在第三问中，则对数据先进行分类，通过对频谱图特征的大致观察与机器学习 SVM 二分法分别对文件进行内容分类，并验证其准确度，选择最佳参数分别为语音/音频文件进行压缩，并说明其原因。

最后对于音频降噪处理，降噪需要对噪声类型进行识别，设置不同的参数对于音频进行降噪处理。这里采用 NLMS（一种改进的自适应滤波算法），能够通过参数设置来对不同噪音的音频进行降噪。将一系列结果进行数据可视化，通过时域图谱可以直观观察音频信号振幅与频率，我们对其图像特征进行总结，并计算了信噪比 SNR 来证明噪音的优化。

5.1 模型优点与特色

（1）本文结合熵权法、灰色关联度、目标优化与建模、SVM 二分类等方法对于音频文件进行压缩，数学模型与要优化的目标充分贴近。

（2）对于音频数据，充分获取了数据的相关信息，并对音频进行频谱可视化呈现，数据直观。

（3）对于多种降噪方法进行分析，选取最为合适本题要求的算法进行求解，并计算 SNR 降噪比来验证是否可行。

5.2 可扩展之处

（1）本文对于音频去噪缺少相应的训练集，若有更多的噪音音频，可以使用卷积神经网络对于样本进行学习，提取其中的 MFCC 或者信号频率，使得对于参数的选择更加准确，参数的设置更能够得到效果好的去噪文件。

（2）本文使用的音频去噪方法对于某一些复杂的音频降噪问题处理仍有提升空间，可以进一步优化算法。

参考文献

- [1]编辑部.回顾二十载家用影音娱乐方式的转捩点音频技术篇[J].家庭影院技术, 2021, (04):50-56.
- [2]赵妍.2024 年全球消费者行为洞察[N].通信产业报,2025-02-24(016).
- [3]王泰然,鲍逸非.基于卷积神经网络和双向长短期记忆网络的微地震记录去噪方法[J/OL].北京大学学报(自然科学版),1-14.
- [4]詹可汉,朱建良.基于 Whisper 模型的机载环境下的语音识别方法研究[J/OL].中国电子科学研究院学报,1-17.
- [5]王峰,李志,邹俊逸.基于熵权法-模糊综合评价的气密性质量评估[J].液压与气动,2025, 49(04):56-62.
- [6]许婕妤,呼斯乐,张金珥.基于 KANO-AHP-熵权法的餐桌适老化设计研究[J/OL].林产工业,1-12.
- [7]张娟,宋维峰,马菁.基于灰色关联度的元阳梯田水源区土壤水分研究[J].亚热带水土保持,2025,37(01):13-18.
- [8]王京城,孙洪兵,刘腾,等.基于特征图谱结合化学计量学与熵权-灰色关联度分析的栽培羌活最佳采收期研究[J/OL].中国中药杂志,1-10.
- [9]白燕燕,白帆,张少哲,等.谱减法在语音信号去噪上的应用[J].家庭影院技术,2024,(02): 59-61.
- [10]樊一帆,张丽丹.强噪环境基于谱减法的录音数字音频信号降噪[J].计算机仿真,2023, 40(11):433-436+474.
- [11]李士林,朱旋,周冬冬. ICEEMDAN-小波阈值法在谐波信号中的应用[J].河北电力技术,2024,43(05):20-26+33.
- [12]庞云鹤.优化自适应滤波算法及其在主动噪声控制系统中的应用[D].烟台大学,2024.

附件 1 代码

4.2.1 三维空间

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# 设置图片清晰度
plt.rcParams['figure.dpi'] = 300
# 设置中文字体
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
# 读取 Excel 文件
excel_file = pd.ExcelFile('/mnt/问题 2 数据.xlsx')
# 获取不同 sheet 的数据
voice_data = excel_file.parse('Sheet1')
music_data = excel_file.parse('Sheet2')
# 定义颜色映射
color_mapping = {'wav': 'red', 'mp3': 'green', 'aac': 'blue'}
# 定义函数绘制不同视角的 3D 图
def plot_3d(data, title, angles):
    fig = plt.figure(figsize=(15, 5))
    for i, angle in enumerate(angles):
        ax = fig.add_subplot(1, 3, i + 1, projection='3d')
        for format_type in data['格式'].unique():
            subset = data[data['格式'] == format_type]
            ax.scatter(subset['大小(KB)'], subset['采样率(Hz)'], subset['比特率'],
c=color_mapping[format_type], label=format_type)
            ax.set_xlabel('大小(KB)')
            ax.set_ylabel('采样率(Hz)')
            ax.set_zlabel('比特率')
            ax.set_title(f'{title}（视角{angle}度）')
            ax.view_init(elev=10, azim=angle)
            ax.legend()
    plt.show()
# 绘制语音数据的 3D 图（三个视角）
plot_3d(voice_data, '语音数据', [0, 45, 90])
# 绘制音乐数据的 3D 图（三个视角）
plot_3d(music_data, '音乐数据', [0, 45, 90])
# 定义函数寻找最优点并计算距离
def find_best_point_and_distances(data):
    # 分别找到大小最小、采样率最大和比特率最大的点
    min_size_point = data[data['大小(KB)'] == data['大小(KB)'].min()].iloc[0] if not
data[data['大小(KB)'] == data['大小(KB)'].min()].empty else None
    max_sample_rate_point = data[data['采样率(Hz)'] == data['采样率(Hz)'].max()].iloc[0]
if not data[data['采样率(Hz)'] == data['采样率(Hz)'].max()].empty else None
```

```

    max_bitrate_point = data[data['比特率'] == data['比特率'].max()].iloc[0] if not
data[data['比特率'] == data['比特率'].max()].empty else None
    # 这里简单返回大小最小的点作为最优点示例，你可以根据实际需求调整
    best_point = min_size_point
    if best_point is not None:
        # 将参与计算的数据转换为 numpy 数组
        data_array = np.array(data[['大小(KB)', '采样率(Hz)', '比特率']])
        best_point_array = np.array(best_point[['大小(KB)', '采样率(Hz)', '比特率']])
        # 检查并确保数组元素都是数值类型
        data_array = pd.to_numeric(data_array.flatten(),
errors='coerce').reshape(data_array.shape)
        best_point_array = pd.to_numeric(best_point_array.flatten(),
errors='coerce').reshape(best_point_array.shape)
        # 计算其余点到最优点的距离
        distances = np.sqrt(((data_array - best_point_array) ** 2).sum(axis=1))
    else:
        distances = None
    return best_point, distances
# 寻找语音数据的最优点并计算距离
voice_best_point, voice_distances = find_best_point_and_distances(voice_data)
# 寻找音乐数据的最优点并计算距离
music_best_point, music_distances = find_best_point_and_distances(music_data)
# 将最优点信息保存到 DataFrame
best_points_df = pd.DataFrame({
    '数据类型': ['语音', '音乐'],
    '最优点大小(KB)': [voice_best_point['大小(KB)'] if voice_best_point is not None else
None,
                        music_best_point['大小(KB)'] if music_best_point is not None
else None],
    '最优点采样率(Hz)': [voice_best_point['采样率(Hz)'] if voice_best_point is not None
else None,
                        music_best_point['采样率(Hz)'] if music_best_point is not
None else None],
    '最优点比特率': [voice_best_point['比特率'] if voice_best_point is not None else None,
                    music_best_point['比特率'] if music_best_point is not None else
None]
})
# 将最优点信息保存到 Excel 文件
best_points_df.to_excel('最优点信息.xlsx', index=False)

print('语音文件最优点信息: ')
print(voice_best_point)
print('音乐文件最优点信息: ')
print(music_best_point)

```

4.2.2 计算灰色关联度

```
import pandas as pd
import numpy as np
from scipy.stats import rankdata
from scipy.optimize import minimize
# 灰色关联分析
def grey_correlation_analysis(reference_seq, comparison_seqs):
    # 数据标准化
    reference_seq = reference_seq / np.mean(reference_seq)
    comparison_seqs = comparison_seqs / np.mean(comparison_seqs, axis=0)
    # 计算差序列
    diff_seqs = np.abs(reference_seq[:, np.newaxis] - comparison_seqs)
    # 计算两极差
    min_diff = np.min(np.min(diff_seqs))
    max_diff = np.max(np.max(diff_seqs))
    # 计算关联系数
    rho = 0.5
    xi = (min_diff + rho * max_diff) / (diff_seqs + rho * max_diff)
    # 计算关联度
    correlation_degrees = np.mean(xi, axis=0)
    return correlation_degrees
# 适应度函数（误差平方和）
def fitness_function(weights, x, y):
    return np.sum((np.dot(x, weights) - y) ** 2)
# 读取文件
excel_file = pd.ExcelFile('点到最优点距离.xlsx')
# 遍历不同工作表（语音和音频）
for sheet_name in excel_file.sheet_names:
    data = excel_file.parse(sheet_name)
    # 提取相关列数据
    X = data[['大小(KB)', '采样率(Hz)', '比特率']].values
    y = data['到最优点的距离'].values
    # 关联度分析
    correlation_degrees = grey_correlation_analysis(y, X)
    print(f'sheet 表名为{sheet_name}的关联度分析结果: ')
    print('大小, 采样率, 比特率与到最优点的距离的关联度分别为:', correlation_degrees)
    # 归一化 X
    X_normalized = (X - np.min(X, axis=0)) / (np.max(X, axis=0) - np.min(X, axis=0))
    # 计算到最优点距离倒数的归一化值
    y_reciprocal = 1 / y
    y_reciprocal[y == 0] = np.max(y_reciprocal)
    y_normalized = (y_reciprocal - np.min(y_reciprocal)) / (np.max(y_reciprocal) -
np.min(y_reciprocal))
    # 使用 minimize 方法求解权重
    bounds = [(0, 1), (0, 1), (0, 1)]
    initial_weights = np.random.rand(3)
```

```

    result = minimize(fitness_function, initial_weights, args=(X_normalized, y_normalized),
bounds=bounds)
    optimal_weights = result.x
    print(f'sheet 表名为{sheet_name}的权重求解结果: ')
    print('大小, 采样率, 比特率的最优权重分别为: ', optimal_weights)
# 给定的新数据
new_data = pd.DataFrame({
    '文件名': ['原始语音_48kHz_24bit.wav', '原始音乐_48kHz_24bit.wav'],
    '格式': ['wav', 'wav'],
    '大小': [839.2285156, 1406.292969],
    '采样率': [48000, 48000],
    '比特率': [1152058, 1152035]
})
# 遍历不同工作表（语音和音频）获取对应权重并给出压缩建议
for sheet_name in excel_file.sheet_names:
    data = excel_file.parse(sheet_name)
    X = data[['大小(KB)', '采样率(Hz)', '比特率']].values
    X_normalized = (X - np.min(X, axis=0)) / (np.max(X, axis=0) - np.min(X, axis=0))
    y = data['到最优点的距离'].values
    y_reciprocal = 1 / y
    y_reciprocal[y == 0] = np.max(y_reciprocal)
    y_normalized = (y_reciprocal - np.min(y_reciprocal)) / (np.max(y_reciprocal) -
np.min(y_reciprocal))
    bounds = [(0, 1), (0, 1), (0, 1)]
    initial_weights = np.random.rand(3)
    result = minimize(fitness_function, initial_weights, args=(X_normalized, y_normalized),
bounds=bounds)
    optimal_weights = result.x
    for index, row in new_data.iterrows():
        file_name = row['文件名']
        size = row['大小']
        sample_rate = row['采样率']
        bitrate = row['比特率']
        # 归一化新数据, 使用之前数据的最值范围
        size_normalized = (size - np.min(X[:, 0])) / (np.max(X[:, 0]) - np.min(X[:, 1]))
        sample_rate_normalized = (sample_rate - np.min(X[:, 1])) / (np.max(X[:, 1]) -
np.min(X[:, 1]))
        bitrate_normalized = (bitrate - np.min(X[:, 2])) / (np.max(X[:, 2]) - np.min(X[:, 2]))
        current_score = optimal_weights[0] * size_normalized + optimal_weights[1] *
sample_rate_normalized + optimal_weights[2] * bitrate_normalized
        # 简单的压缩建议, 假设降低大小和比特率, 提高采样率可能有益
        if optimal_weights[0] > optimal_weights[1] and optimal_weights[0] >
optimal_weights[2]:
            # 大小权重最大, 优先压缩大小
            new_size = size * 0.8 # 假设压缩 20%
            new_sample_rate = sample_rate

```



```

        new_bitrate = bitrate * 0.8 # 假设压缩 20%
    elif optimal_weights[1] > optimal_weights[0] and optimal_weights[1] >
optimal_weights[2]:
        # 采样率权重最大，优先提高采样率
        new_size = size
        new_sample_rate = sample_rate * 1.2 # 假设提高 20%
        new_bitrate = bitrate
    else:
        # 比特率权重最大，优先压缩比特率
        new_size = size
        new_sample_rate = sample_rate
        new_bitrate = bitrate * 0.8 # 假设压缩 20%

    print(f'sheet 表名为{sheet_name}的压缩建议: ')
    print(f'对于文件 {file_name}, 当前得分: {current_score}')
    print(f'建议的新参数: 大小={new_size}, 采样率={new_sample_rate}, 比特率
={new_bitrate}')

```

4.2.2 熵权法-灰色关联度 融合权重与可视化

```

import numpy as np
import matplotlib.pyplot as plt
# 给定的灰色关联度数据
speech_grey = np.array([0.92644523, 0.68681526, 0.9272926])
music_grey = np.array([0.90652193, 0.66512005, 0.90702812])
# 定义熵权法函数
def entropy_weight_method(data):
    m, n = data.shape
    # 归一化数据
    p = data / data.sum(axis=0)
    # 计算熵值，按列计算，所以 axis=0
    e = -(1 / np.log(m)) * np.sum(p * np.log(p + 1e-8), axis=0)
    # 计算权重
    w = (1 - e) / np.sum(1 - e)
    return w
# 示例数据，这里假设从文件中提取数据的方式，实际可根据具体文件结构调整
# 语音数据
speech_data = np.array([
    [33.18359375, 44100, 45557],
    [42.81738281, 44100, 58783],
    [47.18457031, 44100, 64335],
    [54.62792969, 44100, 74997],
    [47.57421875, 16000, 64439],
    [94.32714844, 44100, 128613],
    [93.28613281, 16000, 128058],
    [95.10546875, 16000, 128820],
    [93.28710938, 8000, 128058],

```

```

        [141.4697266, 44100, 192890]
    ])
# 音乐数据
music_data = np.array([
    [78.61328125, 44100, 64200],
    [157.1845703, 44100, 128367],
    [163.3066406, 44100, 133780],
    [158.1054688, 16000, 128492],
    [156.2929688, 8000, 128035],
    [186.2138672, 44100, 152546],
    [186.3564453, 44100, 152663],
    [235.7558594, 44100, 192533],
    [312.5429688, 16000, 256035],
    [392.8984375, 44100, 320867]
])
# 计算熵权法权重
speech_entropy_weights = entropy_weight_method(speech_data)
music_entropy_weights = entropy_weight_method(music_data)
# 对灰色关联度结果进行归一化
speech_grey_normalized = speech_grey / np.sum(speech_grey)
music_grey_normalized = music_grey / np.sum(music_grey)
# 假设将灰色关联度和熵权法结果按照 0.6 和 0.4 的比例合在一起
alpha = 0.6
beta = 0.4
speech_combined = alpha * speech_grey_normalized + beta * speech_entropy_weights
music_combined = alpha * music_grey_normalized + beta * music_entropy_weights
# 设置图片清晰度
plt.rcParams['figure.dpi'] = 300
# 正常显示中文
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
# 正常显示负号
plt.rcParams['axes.unicode_minus'] = False
# 设置柱状图参数
labels = ['大小', '采样率', '比特率']
x = np.arange(len(labels))
width = 0.25
multiplier = 0
# 创建画布
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 6))
# 绘制语音数据的柱状图
for attribute, measurement in zip(['灰色关联度归一化', '熵权法', '综合结果'],
[speech_grey_normalized, speech_entropy_weights, speech_combined]):
    offset = width * multiplier
    rects = ax1.bar(x + offset, measurement, width, label=attribute)
    ax1.bar_label(rects)
    multiplier += 1

```

```

ax1.set_ylabel('权重')
ax1.set_title('语音数据指标权重')
ax1.set_xticks(x + width, labels)
ax1.legend(loc='upper left', bbox_to_anchor=(1, 1))
# 重置 multiplier
multiplier = 0
# 绘制音乐数据的柱状图
for attribute, measurement in zip(['灰色关联度归一化', '熵权法', '综合结果'],
[music_grey_normalized, music_entropy_weights, music_combined]):
    offset = width * multiplier
    rects = ax2.bar(x + offset, measurement, width, label=attribute)
    ax2.bar_label(rects)
    multiplier += 1
ax2.set_ylabel('权重')
ax2.set_title('音乐数据指标权重')
ax2.set_xticks(x + width, labels)
ax2.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()

print("语音数据的熵权法权重: ", speech_entropy_weights)
print("音乐数据的熵权法权重: ", music_entropy_weights)
print("语音数据的灰色关联度归一化结果: ", speech_grey_normalized)
print("音乐数据的灰色关联度归一化结果: ", music_grey_normalized)
print("语音数据的综合结果: ", speech_combined)
print("音乐数据的综合结果: ", music_combined)

```

4.3.1 MFCC,Mel,ZCR 信息提取

```

import os
import numpy as np
import librosa
import pandas as pd
# 特征提取函数（沿用之前定义的）
def extract_mfcc(audio_path, n_mfcc=40):
    y, sr = librosa.load(audio_path, sr=44100)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc, n_fft=2048, hop_length=512)
    return np.mean(mfcc, axis=1)
def extract_mel_spectrogram(audio_path, n_mels=128):
    y, sr = librosa.load(audio_path, sr=44100)
    mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512,
n_mels=n_mels)
    return np.mean(np.log1p(mel_spec), axis=1)
def extract_zcr(audio_path):
    y, sr = librosa.load(audio_path, sr=44100)
    zcr = librosa.feature.zero_crossing_rate(y, frame_length=2048, hop_length=512)
    return np.mean(zcr)

```

```

# 加载数据并提取特征
data_dir = "附件 1" # 请确保路径正确
features_list = []
filenames_list = []
for filename in os.listdir(data_dir):
    if not filename.endswith(('wav', 'aac', 'mp3')):
        continue
    path = os.path.join(data_dir, filename)
    filenames_list.append(filename)
    mfcc = extract_mfcc(path)
    mel = extract_mel_spectrogram(path)
    zcr = extract_zcr(path)
    # 整理特征 (MFCC 40 维 + 梅尔频谱 128 维 + 过零率 1 维)
    features = np.concatenate([mfcc, mel, [zcr]])
    features_list.append(features)
# 创建 DataFrame 并保存到 Excel
columns = [f"MFCC_{i}" for i in range(40)] + [f"Mel_{i}" for i in range(128)] + ["ZCR"]
df = pd.DataFrame(features_list, columns=columns)
df.insert(0, "Filename", filenames_list)
# 保存到 Excel
excel_path = "audio_features.xlsx"
df.to_excel(excel_path, index=False)
print(f'音频特征已保存至 {excel_path}')

```

4.3.2 SVM 二分法

```

import os
import numpy as np
import librosa
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
# 特征提取函数
def extract_mfcc(audio_path, n_mfcc=40):
    y, sr = librosa.load(audio_path, sr=44100)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc, n_fft=2048, hop_length=512)
    return np.mean(mfcc, axis=1)

def extract_mel_spectrogram(audio_path, n_mels=128):
    y, sr = librosa.load(audio_path, sr=44100)
    mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512,
n_mels=n_mels)

```

```

    return np.mean(np.log1p(mel_spec), axis=1)

def extract_zcr(audio_path):
    y, sr = librosa.load(audio_path, sr=44100)
    zcr = librosa.feature.zero_crossing_rate(y, frame_length=2048, hop_length=512)
    return np.mean(zcr)

# 数据加载与划分
def load_data(data_dir):
    X, y = [], []
    for filename in os.listdir(data_dir):
        if not filename.endswith(('.wav', '.aac', '.mp3')):
            continue
        path = os.path.join(data_dir, filename)
        label = 0 if '语音' in filename else 1 # 假设文件名含'voice'或'music'
        mfcc = extract_mfcc(path)
        mel = extract_mel_spectrogram(path)
        zcr = extract_zcr(path)
        features = np.concatenate([mfcc, mel, [zcr]])
        X.append(features)
        y.append(label)
    return np.array(X), np.array(y)

# 模型训练与评估
def train_svm_classifier(X_train, y_train):
    pipeline = make_pipeline(
        StandardScaler(),
        SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
    )
    pipeline.fit(X_train, y_train)
    return pipeline

# 可视化函数
def visualize_features(X, y, title):
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)

    plt.figure(figsize=(10, 6))
    sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette={0: 'blue', 1: 'orange'},
alpha=0.8)
    plt.title(title)
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.legend(title="Class", labels=["Voice", "Music"])
    plt.show()

```

```

def plot_confusion_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=["Voice", "Music"], yticklabels=["Voice", "Music"])
    plt.title(title)
    plt.xlabel("Predicted Class")
    plt.ylabel("True Class")
    plt.show()

def plot_decision_boundary(clf, X, y):
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)
    h = 0.2 # 网格步长
    x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
    y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = clf.predict(pca.inverse_transform(np.c_[xx.ravel(), yy.ravel()]))
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(10, 6))
    plt.contourf(xx, yy, Z, alpha=0.8)
    sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette={0: 'blue', 1: 'orange'},
alpha=0.8)
    plt.title("Decision Boundary (PCA Reduced)")
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.legend(title="Class", labels=["Voice", "Music"])
    plt.show()

# 主函数
if __name__ == "__main__":
    data_dir = "附件 1" # 请确保路径正确
    X, y = load_data(data_dir)
    # 划分训练集和测试集, 训练集 30%, 测试集 70%
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7, random_state=42)
    print(f"Training set samples: {len(X_train)}, Test set samples: {len(X_test)}")
    clf = train_svm_classifier(X_train, y_train)
    y_train_pred = clf.predict(X_train)
    y_test_pred = clf.predict(X_test)
    print("\nTraining Set Evaluation")
    print(f"Training set accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
    print("Classification Report:")
    print(classification_report(y_train, y_train_pred, target_names=["Voice", "Music"]))
    print("\nTest Set Evaluation ")
    print(f"Test set accuracy: {accuracy_score(y_test, y_test_pred):.4f}")
    print("Test set classification results:", y_test_pred)
    visualize_features(X_train, y_train, "Training Set Feature Distribution (PCA Reduced)")

```

```

plot_confusion_matrix(y_train, y_train_pred, "Training Set Confusion Matrix")
plot_decision_boundary(clf, X_train, y_train)

```

4.3.2 获取频谱

```

import os
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

def plot_spectrum(audio_path, output_path):
    try:
        y, sr = librosa.load(audio_path)
    except Exception as e:
        print(f'Error loading {audio_path} with librosa: {e}')
        return
    # 调整快速傅里叶变换窗口大小（影响频率分辨率）和帧移（影响时间分辨率）
    n_fft = 2048 # 增大窗口大小，提高频率分辨率
    hop_length = 512 # 调整帧移，控制时间与频率分辨率的平衡
    D = librosa.stft(y, n_fft=n_fft, hop_length=hop_length)
    S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)
    plt.figure(figsize=(12, 6))
    # 使用线性频率轴（更直观展示频率差异，默认是对数轴会压缩高频细节）
    librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='linear',
hop_length=hop_length)
    plt.colorbar(format='%+2.0f dB')
    plt.title('Spectrum')
    plt.xlabel('Time (s)')
    plt.ylabel('Frequency (Hz)')
    plt.tight_layout()
    plt.savefig(output_path)
    plt.close()

def process_audio_folder(folder_path, output_folder):
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
    for root, dirs, files in os.walk(folder_path):
        for file in files:
            if file.endswith(('wav', 'aac', 'mp3')):
                audio_path = os.path.join(root, file)
                file_name = os.path.splitext(file)[0]
                output_path = os.path.join(output_folder, f'{file_name}_spectrum.png')
                plot_spectrum(audio_path, output_path)

if __name__ == "__main__":
    audio_folder = "附件 1"
    output_folder = "附件 1 频谱 2" # 保存到新文件夹
    process_audio_folder(audio_folder, output_folder)

```

4.4.2 NLMS 实例结果可视化

```
import numpy as np
import matplotlib.pyplot as plt
# 设置随机种子以确保结果可重复
np.random.seed(42)
# 参数设置
num_samples = 48000 # 2 秒音频，假设采样率为 24000Hz
sample_rate = 24000
time = np.linspace(0, num_samples / sample_rate, num_samples)
# 生成带状噪声（均匀分布噪声，振幅随机变化）
def generate_striped_noise(num_samples):
    # 生成基底噪声
    noise = np.random.uniform(-1, 1, num_samples)
    # 随机选择部分区域增强噪声，模拟带状特性
    for _ in range(20): # 添加 20 个随机带状区域
        start = np.random.randint(0, num_samples)
        length = np.random.randint(100, 1000) # 每个带状区域长度在 100-1000 个采样
点之间
        if start + length > num_samples:
            break
        noise[start:start + length] *= np.random.uniform(1.5, 3) # 振幅增强 1.5-3 倍
    return noise
# 生成突发噪声（随机间隔的脉冲噪声，限制振幅）
def generate_impulsive_noise(num_samples):
    noise = np.zeros(num_samples)
    # 随机选择突发位置
    num_impulses = np.random.randint(100, 200) # 突发次数在 100-200 次之间
    impulse_positions = np.random.choice(num_samples, size=num_impulses,
replace=False)
    # 为每个突发位置生成不同强度和宽度的脉冲，限制振幅不超过 1.5
    for pos in impulse_positions:
        intensity = np.random.uniform(0.5, 1.5) # 脉冲强度在 0.5-1.5 之间，确保不超过
纵坐标最大值
        width = np.random.randint(1, 20) # 脉冲宽度在 1-20 个采样点之间
        noise[max(0, pos - width // 2):min(num_samples, pos + width // 2 + 1)] += intensity
    return noise
# 生成背景噪声（低频+高频成分，振幅缓慢变化）
def generate_background_noise(num_samples):
    # 低频成分
    low_freq = np.sin(2 * np.pi * 0.1 * time) # 0.1Hz 正弦波
    # 高频成分
    high_freq = np.random.normal(0, 0.3, num_samples) # 高斯白噪声，标准差 0.3
    # 振幅缓慢变化
    amplitude_modulation = np.linspace(0.5, 1.5, num_samples) # 振幅从 0.5 线性增长
到 1.5
    noise = (low_freq + high_freq) * amplitude_modulation
```



```

        return noise
# 生成噪声信号
striped_noise = generate_striped_noise(num_samples)
impulsive_noise = generate_impulsive_noise(num_samples)
background_noise = generate_background_noise(num_samples)
# 绘制时域波形图
plt.figure(figsize=(12, 8))
# 带状噪声
plt.subplot(3, 1, 1)
plt.plot(time[:1000], striped_noise[:1000]) # 绘制前 1000 个采样点以清晰显示细节
plt.title('带状噪声 (前 1000 个采样点)')
plt.xlabel('时间 (s)')
plt.ylabel('振幅')
plt.ylim(-1.5, 1.5) # 设置纵坐标范围
plt.grid(True)
# 突发噪声
plt.subplot(3, 1, 2)
plt.plot(time[:1000], impulsive_noise[:1000]) # 绘制前 1000 个采样点以清晰显示细节
plt.title('突发噪声 (前 1000 个采样点)')
plt.xlabel('时间 (s)')
plt.ylabel('振幅')
plt.ylim(-1.5, 1.5) # 设置纵坐标范围
plt.grid(True)
# 背景噪声
plt.subplot(3, 1, 3)
plt.plot(time[:1000], background_noise[:1000]) # 绘制前 1000 个采样点以清晰显示细节
plt.title('背景噪声 (前 1000 个采样点)')
plt.xlabel('时间 (s)')
plt.ylabel('振幅')
plt.ylim(-1.5, 1.5) # 设置纵坐标范围
plt.grid(True)
plt.tight_layout()
plt.savefig('noise_waveforms_updated_impulsive.png') # 保存波形图到当前目录
plt.show()
# 打印前几个采样点的噪声值示例
print("带状噪声 (前 10 个采样点):", striped_noise[:10])
print("突发噪声 (前 10 个采样点):", impulsive_noise[:10])
print("背景噪声 (前 10 个采样点):", background_noise[:10])

```

4.4.2 NSML 滤波器（参数可调整）

```

import numpy as np
import soundfile as sf
def nlms_denoise(input_signal, filter_length, mu, noise_threshold):
    # 初始化滤波器权重
    weights = np.zeros(filter_length)
    # 初始化输出信号

```

```

output_signal = np.zeros(len(input_signal))
# 初始化参考信号
reference_signal = np.zeros(filter_length)
for n in range(filter_length, len(input_signal)):
    # 检测是否有噪声
    if np.abs(input_signal[n]) > noise_threshold:
        # 如果有噪声，更新参考信号
        reference_signal = input_signal[n - filter_length:n]
    # 提取当前输入信号帧
    x = reference_signal
    # 计算滤波器输出
    y = np.dot(weights, x)
    # 计算误差
    e = input_signal[n] - y
    # 归一化因子
    norm_factor = np.dot(x, x) + 1e-10 # 加一个小正数防止除零
    # 更新滤波器权重
    weights = weights + (mu * e * x) / norm_factor
    # 保存去噪后的信号
    output_signal[n] = e
return output_signal
# 读取音频文件
input_audio, sample_rate = sf.read("part1.wav")
# 如果音频是立体声，转换为单声道
if len(input_audio.shape) > 1:
    input_audio = np.mean(input_audio, axis=1)
# 可以尝试不同的参数组合
filter_lengths = [128, 256, 512]
mus = [0.01, 0.1, 0.5]
noise_thresholds = [0.1, 0.5, 1]
for fl in filter_lengths:
    for m in mus:
        for nt in noise_thresholds:
            # 进行去噪处理
            denoised_audio = nlms_denoise(input_audio, fl, m, nt)
            # 保存去噪后的音频，文件名包含参数信息
            filename = f"part1_denoised_nlms_fl{fl}_mu{m}_nt{nt}.wav"
            sf.write(filename, denoised_audio, sample_rate)

```

4.4.2 数据可频谱图可视化

```

import numpy as np
import soundfile as sf
import matplotlib.pyplot as plt
def calculate_snr(original, denoised):
    """
    计算原始音频和去噪后音频的信噪比

```

```

:param original: 原始音频信号
:param denoised: 去噪后的音频信号
:return: 信噪比 (dB)
"""

noise = original - denoised
signal_power = np.mean(np.square(original))
noise_power = np.mean(np.square(noise))
return 20 * np.log10(signal_power / noise_power)
# 读取原始音频文件
original_audio, original_sample_rate = sf.read("part2.wav")
# 如果音频是立体声，转换为单声道
if len(original_audio.shape) > 1:
    original_audio = np.mean(original_audio, axis=1)
# 读取去噪后的音频文件
denoised_audio, denoised_sample_rate = sf.read("part2_denoised.wav")
# 如果音频是立体声，转换为单声道
if len(denoised_audio.shape) > 1:
    denoised_audio = np.mean(denoised_audio, axis=1)
# 计算信噪比
snr = calculate_snr(original_audio, denoised_audio)
print(f'原始文件 part1.wav 与去噪后文件 part1_denoised.wav 的信噪比 (SNR): {snr:.2f} dB')
# 绘制时域波形图
plt.figure(figsize=(12, 6))
# 原始音频波形
plt.subplot(2, 1, 1)
plt.plot(np.arange(len(original_audio)) / original_sample_rate, original_audio)
plt.title('Original Audio')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
# 去噪后音频波形
plt.subplot(2, 1, 2)
plt.plot(np.arange(len(denoised_audio)) / denoised_sample_rate, denoised_audio)
plt.title('Denoised Audio')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

附录 2 音频排序完整版

附录表 1 语音文件排序

文件名	格式	大小(KB)	采样率(Hz)	比特率	到最优点的距离
语音_44100Hz_AAC_96kbps.aac	aac	33.18359375	44100	45557	0
语音_44100Hz_AAC_128kbps.aac	aac	42.81738281	44100	58783	13226.00351
语音_44100Hz_MP3_64kbps.mp3	mp3	47.18457031	44100	64335	18778.00522
语音_44100Hz_AAC_192kbps.aac	aac	54.62792969	44100	74997	29440.00781
语音_16000Hz_MP3_64kbps.mp3	mp3	47.57421875	16000	64439	33854.69142
语音_44100Hz_MP3_128kbps.mp3	mp3	94.32714844	44100	128613	83056.02251
语音_16000Hz_8bit.wav	wav	93.28613281	16000	128058	87155.19843
语音_16000Hz_MP3_128kbps.mp3	mp3	95.10546875	16000	128820	87876.85135
语音_8000Hz_16bit.wav	wav	93.28710938	8000	128058	90053.47641
语音_44100Hz_MP3_192kbps.mp3	mp3	141.4697266	44100	192890	147333.0398
语音_16000Hz_16bit.wav	wav	186.5292969	16000	256058	212368.3228
语音_44100Hz_MP3_320kbps.mp3	mp3	235.7558594	44100	321448	275891.0744
语音_44100Hz_8bit.wav	wav	257.0439453	44100	352858	307301.0815
语音_22050Hz_16bit.wav	wav	257.0449219	22050	352858	308091.1508
语音_32000Hz_16bit.wav	wav	373.015625	32000	512058	466658.0209
语音_44100Hz_16bit.wav	wav	514.0449219	44100	705658	660101.1751
语音_48000Hz_16bit.wav	wav	559.5	48000	768058	722511.7176
语音_48000Hz_24bit.wav	wav	839.2285156	48000	1152058	1106508.167

附录表 2 音乐文件排序

文件名	格式	大小(KB)	采样率(Hz)	比特率	到最优点的距离
音乐_44100Hz_MP3_64kbps.mp3	mp3	78.61328125	44100	64200	0
音乐_44100Hz_MP3_128kbps.mp3	mp3	157.1845703	44100	128367	64167.0481
音乐_44100Hz_AAC_128kbps.aac	aac	163.3066406	44100	133780	69580.05154
音乐_16000Hz_MP3_128kbps.mp3	mp3	158.1054688	16000	128492	70164.64625
音乐_8000Hz_16bit.wav	wav	156.2929688	8000	128035	73335.68885
音乐_44100Hz_AAC_192kbps.aac	aac	186.2138672	44100	152546	88346.06553
音乐_44100Hz_AAC_256kbps.aac	aac	186.3564453	44100	152663	88463.06561
音乐_44100Hz_MP3_192kbps.mp3	mp3	235.7558594	44100	192533	128333.0962
音乐_16000Hz_16bit.wav	wav	312.5429688	16000	256035	193882.2631
音乐_44100Hz_MP3_320kbps.mp3	mp3	392.8984375	44100	320867	256667.1924
音乐_44100Hz_8bit.wav	wav	430.7070313	44100	352835	288635.2148
音乐_22050Hz_16bit.wav	wav	430.7070313	22050	352835	289476.2334
音乐_32000Hz_16bit.wav	wav	625.0429688	32000	512035	447998.7676
音乐_44100Hz_16bit.wav	wav	861.3710938	44100	705635	641435.4776
音乐_48000Hz_16bit.wav	wav	937.5429688	48000	768035	703846.3291
音乐_48000Hz_24bit.wav	wav	1406.292969	48000	1152035	1087842.801